Soft Computing (2026) 30:2109–2133
https://doi.org/10.1007/s00500-025-10734-5

FOCUS

Effective resistance and kernel-based graph sparsification for community detection in complex networks

Annalisa Socievole[1] · **Clara Pizzuti**[1]

Accepted: 25 April 2025 / Published online: 18 February 2026

Abstract
In community detection tasks, accurately measuring the distance between nodes is fundamental for correctly grouping the nodes according to their similarity. This work proposes a community detection method exploiting the *effective resistance*, a measure derived from the field of electrical circuits that has been shown to be a good distance metric for uncovering communities. In the pre-processing phase, the method weights the input graph G with the effective resistance between each couple of nodes. Then, to turn this distance into a similarity value reflecting the closeness between each couple of nodes, different kernel functions for the effective resistance are tested. Finally, since many of the networks we deal with are not sparse, G is reduced to a smaller graph G' by cutting a percentage of less important edges through a *weight thresholding* sparsification procedure. The amount of edge cuts is chosen evaluating the spectral similarity between G and G'. In such a way, we are able to work with fewer edges gaining in computational time and storage resources. At the end of the pre-processing phase, we run on the sparsified graph G' a community detection method based on Genetic Algorithms maximizing the weighted modularity as objective function. Experimenting on both real-world and synthetic networks, we demonstrate the effectiveness of such approach when compared to other benchmarks.

Keywords Community detection · Graph sparsification · Genetic algorithm · Effective resistance · Moore-Penrose pseudoinverse

1 Introduction

Complex networks Van Mieghem (2011) are widely used to model the dynamics of real-world systems, such as social, telecommunication, brain, biological, and financial networks. These systems are represented as *graphs*, where nodes denote system units (e.g., users, brain regions, devices) and edges represent their connections. A common feature of these systems is the presence of dense graphs Barrat et al. (2004), which pose computational challenges. Processing dense graphs with tools of network science is costly and sometimes impossible, unless some pre-processing techniques cutting the "less important" edges or weak connections are applied aiming at reducing the size of the edge set. As a result, the algorithms run faster and the graphs can be stored more compactly.

Various *edge pruning* or *graph sparsification* methods Hashemi et al. (2024) have been proposed in recent years (Fung et al. 2011; Spielman and Srivastava 1996; Dianati 2016; Coscia and Neffke 2017; Kobayashi et al. 2019; Arora and Upadhyay 2019; Coscia and Rossi 2019; Yu et al. 2022; Li et al. 2022; Liu et al. 2023; Jambulapati et al. 2024; Zhang et al. 2024; Su et al. 2025). Sparsification produces a subgraph, or *sparsifier*, $G' = (V, W)$, similar to the original graph $G = (V, E)$ according to specific metrics. A common approach is *weight thresholding*, which removes edges below a threshold while preserving community structures Yan et al. (2018). In Spielman and Srivastava (1996), edges are included in G' based on their *effective resistance*, a measure related to random spanning trees and commute times (Klein and Randić 1993; Chandra et al. 1996; Saerens et al. 2004).

✉ Annalisa Socievole
annalisa.socievole@icar.cnr.it

Clara Pizzuti
clara.pizzuti@icar.cnr.it

[1] Institute for High Performance Computing and Networking (ICAR), National Research Council of Italy (CNR), Via Pietro Bucci, 8-9C, Rende (CS) 87036, Italy

This work focuses on effective resistance-based sparsification for *community detection*, which aims to partition nodes into groups based on similarity. Our approach produces a sparsifier G' that retains the community structure of G, leveraging effective resistance for grouping nodes (Yen et al. 2009; Sommer et al. 2016; Pizzuti and Socievole 2021a, b; Socievole and Pizzuti 2023a, b). In Pizzuti and Socievole (2021a), we introduced *OmeGAnet*, a *Genetic Algorithm (GA)* Goldberg (1989) that uses effective resistance for edge weighting and sparsification. The GA operates on G' by retaining edges with the lowest effective resistance values and optimizes modularity Girvan and Newman (2002). Later, we proposed *OmeGAnet+* Socievole and Pizzuti (2023b), which improves sparsification by removing edges based on the *minimum absolute spectral similarity (MASS)* Yan et al. (2018), selecting the sparsification level that maximizes MASS while preserving graph connectivity. In a similar manner but more effective, in this paper we exploit a GA for community detection. It is worth noting that in the context of community detection, Graph Neural Networks (GNNs) have also emerged as powerful tools for processing graph-structured data, leveraging both node features and structural information through message-passing mechanisms (Kipf and Welling 2017; Hamilton et al. 2017), and offering scalability and adaptability to large datasets. For community detection, GNNs utilize node embeddings to cluster nodes based on their structural and feature-based similarities (Zhang et al. 2019; Chen et al. 2021). Methods like Graph Convolutional Networks (GCNs) and Graph Autoencoders, for example, have been successfully applied to identify communities by optimizing embeddings for clustering tasks (Wang et al. 2019; Li et al. 2020). However, their performance is heavily influenced by model architecture, hyperparameter tuning, and computational demands, which can become prohibitive for extremely large networks. In contrast, Genetic Algorithm (GA)-based methods, like the one proposed in this work, focus on directly optimizing global quality metrics such as modularity (Girvan and Newman 2002). This targeted approach makes GA-based methods computationally efficient and interpretable, especially for structural networks without node features. Unlike GNNs, GA-based methods provide direct community assignments rather than intermediate embeddings, allowing for a more transparent evaluation of results. While GNNs excel in large, feature-rich graphs, GA-based methods are better suited for moderate-sized, purely structural networks, offering robust and interpretable solutions.

A challenge with effective resistance sparsification is ensuring meaningful distance/similarity values in the adjacency matrix. *Kernels* on graphs Avrachenkov et al. (2019) are able to offer an effective solution by transforming the input graph into a symmetric positive semidefinite matrix K that better separates nodes in Euclidean space. In such a way, kernel-based methods are able to improve clustering performance as shown in various applications (Yen et al. 2009; Sommer et al. 2016, 2017; Aynulin 2019a, b; Pizzuti and Socievole 2022; Zheng et al. 2023). Inspired by previous work on kernel-based clustering and graph sparsification, in this work we extend and improve our earlier work Socievole and Pizzuti (2023b) by proposing *KOmeGAnet+*, a community detection method which relies on two steps:

(1) first, weight the input graph G by computing the effective resistance distance between each couple of nodes, define a similarity or kernel matrix K from the adjacency matrix of G and subsequently compute a sparsified matrix K' from K through the MASS index;
(2) then, run over K' a GA optimizing modularity as fitness function in order to cluster the nodes of the graph.

The effective resistance and kernel-based community detection algorithm proposed in this paper differs from existing related works by the fact that *kernels* and *sparsification* are jointly used by taking advantage by both techniques in community detection. Besides that, we also propose to structure the pre-processing step (step 1) by first weighting the input graph with the effective resistance, then applying kernels and finally sparsifying. The rationale behind this order is that the weighting procedure and kernels should precede sparsification and not vice-versa in order to first have a clear vision of node similarity based on effective resistance and kernels, and than exploit this information to better drive sparsification thus avoiding loosing the underlying community structure of the graph. Compared to our previous conference work Socievole and Pizzuti (2023b), the main extensions in this paper concern:

- a comprehensive literature review on the three main topics of the paper: (1) graph sparsification, (2) effective resistance based community detection, and (3) kernel-based community detection;
- the application of kernels to the adjacency matrix of the input graph and the evaluation of their impact on community detection;
- the jointly use of kernels and effective resistance based graph sparsification for community detection;
- a thorough redesign of all the experiments including the investigation of additional algorithms and larger networks with different edge densities.

We primarily investigate the extent to which kernels improve effective resistance based community detection performance by making an extensive experimental comparison between different algorithms over both real-world

and synthetic datasets. Specifically, we first compare several classes of kernels (*adjacency matrix* based, *Laplacian* based and *Markov matrix* based) focusing on a basic GA which runs on the adjacency matrix of G weighted through the effective resistance. Differently from previous works comparing kernels on the unweighted adjacency matrix of G, here we particularly analyze their impact when the input of the method is the effective resistance based kernel matrix. The results of this preliminary analysis show that the adjacency-based kernels are the most suitable over real-world networks, and synthetic networks with a non-random structure of communities. On the contrary, Laplacian or Markov matrix based kernels are needed over synthetic networks with unclear community structure. In a second experiment, we compare kernel-based and non kernel-based community detection methods schemes and evaluate the role of the kernels over methods based on GAs since our proposal is based on this evolutionary method. It is shown that the kernel and effective resistance based GA performs the best in most of the cases, while when the graph edge density is high the contestant method using sparsification performs better. In a third experiment, we finally demonstrate the benefits of using kernels in conjunction with sparsification especially when the networks are highly dense, by evaluating *KOmeGAnet+*.

Besides the introduction, the paper is structured as follows. Section 2 reviews related work on graph sparsification, effective resistance based community detection, and kernel-based community detection. Section 3 briefly recalls some basic concepts of the theory of effective resistance, MASS and kernels. Section 4 describes the proposed community detection method. Section 5 introduces the experimental setup and Section 6 illustrates and discusses the results of the various experiments. Section 7 concludes the paper.

2 Related work

This section provides a description of the related work organized into the three main topics of the paper: (a) graph sparsification techniques, (b) community detection algorithms based on effective resistance similarity and (c) kernel-based community detection.

2.1 Graph sparsification

Graph reduction techniques are increasingly gaining prominence due to their ability to simplify the analysis, the computation and the storage of large graphs while preserving their essential properties. In a recent work, Hashemi et al. (2024) present a comprehensive survey on graph reduction methods, including *graph sparsification*, *graph coarsening*, and *graph condensation*. Graph sparsification, which is the candidate graph reduction technique of the present paper, was introduced by Benczúr and Karger (1996) to accelerate cut algorithms whose running time depends on the number of edges. This technique deals with the approximation of a graph by keeping a subset of its edges and its vital vertices. On the contrary, graph coarsening groups nodes into supernodes, with original edges spanning across groups being aggregated into super-edges using a specified aggregation algorithm. Differently from the aforementioned two reduction methods, graph condensation reduces a graph by synthesizing a smaller graph while preserving the performance of GNNs.

Yan et al. (2018) propose weight thresholding, a simple graph sparsification technique that aims at reducing the edges of a graph by cutting all the edges that have a weight below a threshold. By analyzing several synthetic and real-world networks, the analysis shows that while local and global properties of the network are lost under edge removals, the organization in communities is kept. This is due to the correlation between weight and degree according to which high weights are typical of links attached to high-degree nodes. For this reason, in the present work dealing with community detection, we take into consideration this graph reduction technique.

Coscia and Neffke (2017) extract a significant edge backbone from a noisy complex network by proposing a network backboning algorithm which takes as input a dense and noisy network and returns a reduced version of it. The approach, called Noise-Corrected (NC) backbone cuts spurious connections analyzing their edge weight: the backboning algorithm decide whether to prune an edge or not comparing the weight to a threshold δ. In a later work, Coscia and Rossi (2019) project a bipartite network into a unipartite weighted graph and then exploit the aforementioned backboning technique to extract only the meaningful edges.

Effective resistance based graph sparsification is proposed by Spielman and Srivastava in a seminal work (Spielman and Srivastava 2008) as a nearly-linear time algorithm that produces high-quality sparsifiers. The sparsifier is a subgraph of the input graph which is computed in $O(m)$ time through a random sampling that keeps an edge with a sampling probability given by the effective resistances of the edge.

Mercier et al. (2022) sparsify a mobility network where most locations have a large number of links to many other locations, for simplifying the simulation of pandemics. Two types of sparsification methods are applied: weight thresholding and edge sampling by effective resistance Spielman and Srivastava (2008). The resulting sparse network is able to preserves both local (i.e. probability of infection or the

distribution of the arrival times of a node) and global aspects (i.e. the epidemic size) of a SIR model.

Note that in the present work, we use weight thresholding sparsification technique and not edge sampling Spielman and Srivastava (2008) but differently form Yan et al. (2018) the thresholding is applied over a graph weighted with the effective resistance.

2.2 Effective resistance based community detection

In Lu et al. (2012), Lu et al. exploit for the first time the resistance distance for clustering a network. Since each community has typically some high degree central nodes, according to the properties of small-world and scale-free networks, the method decides which are the central nodes using their degree and the resistance distance. The node with the largest degree is put at the center of the first community, the second high-degree node in the second community and so on. However, considering that two center nodes may be in the same community, the network is partitioned using both the resistance distance and the degree of nodes. Simulations on synthetic networks and the real-world Zachary Karate Club and Bottlenose Dolphins networks show that the method is able to match the ground-truth.

Zhang and Bu (2019) detect the community structure in complex networks utilizing a Gaussian function of the resistance distance for weighting the input graph and a bisection spectral method for dividing the nodes into communities. The bisection procedure is repeated until the underlying number of communities is achieved. Experiments on the Zachary Karate Club, the Bottlenose Dolphins, and the American College Football networks demonstrate that the method is stable and reliable. The proposed method, however, presents two drawbacks: the high time complexity making it unpractical for large networks and the need to specify in advance the number of communities to search.

In a later work, Gancio and Rubido (2022) present an unsupervised community detection algorithm which adapts Zhang and Bu's algorithm Zhang and Bu (2019) to detect communities without specifying a-priori the number of communities. Modularity optimization is added to the spectral partitioning in order to iterate the algorithm without fixing the number of communities or control its outcomes in advance. To evaluate the accuracy of the method, Girvan-Newman (GN) and Lancichinetti-Fortunato-Radicchi (LFR) Lancichinetti et al. (2008) benchmark networks with 128 and 1000 nodes are used. The experimental results show the high accuracy of the method.

In a recent work Socievole and Pizzuti (2023a), *SGAnet* is presented as an extension of *OmeGAnet* that computes a *degree normalized effective resistance* between nodes. Since in some cases, the effective resistance values of edges across communities may be *smaller* than the effective resistance values of edges within communities (contradicting the property of this distance measure of being lower for intra-community edges Rubido et al. (2013)), normalizing the elements of the effective resistance matrix according to the maximum degree of the end nodes helps in better distinguishing the bridge edges.

It is worth pointing out that the substantial difference between our proposal and the aforementioned effective resistance based community detection methods is how this metric is taken into account: (1) the effective resistance is used to weight edges and also to cut unnecessary and less important edges, and (2) kernels are jointly used for a further "refinement" of the metric.

2.3 Kernel-based community detection

Kernel-based community detection is often referred to as kernel clustering or similarity-based kernel clustering. A survey on kernel clustering can be found in Filippone et al. (2008) and a thorough description of similarity-based approaches in Yen et al. (2009). It is worth pointing out that the present work deals with clustering the nodes of a graph and not data[1]. The primary objective of kernel-based community detection algorithms is to define a meaningful similarity/distance measure between each couple of connected nodes in order to group them into communities. In the literature, the similarity between nodes is often referred to as *relatedness* or *proximity*, even if in this last case some conditions need to be met to be a proximity measure Chebotarev and Shamis (2006). The similarity matrix is often defined as a positive semi-definite matrix called *kernel matrix*. Once these similarities between every pair of nodes are computed, they can be used for partitioning the nodes accordingly (i.e. similar nodes are grouped together).

Yen et al. (2007) use the Sigmoid Commute-Time kernel to cluster the nodes of a weighted graph. In a pre-processing procedure, the commute-time kernel is computed on the adjacency matrix of the graph and then the sigmoid transformation is applied thus producing a kernel matrix. Related to the effective resistance, the commute-time kernel takes its name from the average *commute time* between two nodes i and j (i.e. the average number of steps a random walker takes for going from i to j and returning) which can be computed through the Moore Penrose pseudoinverse of the Laplacian matrix. Then, the method clusters the nodes by running the k-means and a fuzzy k-means on the kernel matrix. The results show that the joint use of commute-time kernel and kernel clustering outperform standard k-means, as well as spectral clustering, on a difficult document

[1] The term *kernel clustering* in its classical meaning refers to data clustering.

clustering problem. A comparative study on kernel-based clustering with the sigmoid commute-time kernel can be also found in Yen et al. (2009).

In Sommer et al. (2016), Sommer et al. evaluate 6 different kernels on clustering tasks by comparing two versions of a distance based kernel *k*-means with the Louvain method Blondel et al. (2008), for a total of 13 different clustering methods. The experiments are run over several graphs, including the Zackary Karate Club, the American College Football, and three synthetically generated LFR networks. The results show that Louvain is outperformed by the kernel-based methods and the Free Energy and the Randomized Shortest-Path kernels show an edge over the others.

Aynulin (2019a) considers the efficiency of several kernels in clustering networks when the topology varies in terms of network size, cluster structure, degree distribution, edge density and so on. Specifically, the work analyzes the performance of a given kernel over different network topologies demonstrating that kernels are generally topology-dependent, however, the Walk and the Communicability kernels behave well for most topologies.

Note that differently from the aforementioned works, the present work computes the kernels on the adjacency matrix of the input graph weighted with the effective resistance.

3 Theory review

In this section, we briefly introduce the mathematical notation and the basic theory behind *effective resistance*, *MASS*, and *kernels*, on which the proposed community detection method is built.

3.1 Effective resistance

The *effective resistance* is a measure derived from the field of electrical circuits, which is often applied to networks for its interesting properties. If considered as an electric circuit, a network can be seen as composed by nodes connected through resistors for which the resistance is a global distance since it takes into account both the direct and the indirect connections between the nodes. The more the paths between two nodes, the lower this distance. Given an undirected and connected graph $G = (V, E)$, we can assign to G an *equivalent electric network* where each edge $(i, j) \in E$ has a weight w_{ij} representing its conductance, that is the inverse of the electrical resistance ω_{ij} of a resistor, so that $\omega_{ij} = \frac{1}{w_{ij}}$ *ohm* Klein and Randić (1993).

A distance value can be associated with each edge of the graph as follows (Klein and Randić 1993; Van Mieghem et al. 2017). Let A be the adjacency matrix of G where the element $a_{ij} = 1$ if there is an edge between nodes i and j, $\Delta = diag(d_i)$ the $N \times N$ diagonal degree matrix, where $d_i = \sum_{j=1}^{N} a_{ij}$, L the Laplacian matrix of the graph G defined as the $N \times N$ symmetric matrix $L = \Delta - A$, with elements

$$l_{ij} = \begin{cases} d_i & \text{if } i = j \\ -1 & \text{if the edge } (i, j) \in E \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

The effective resistance ω_{ij} is the voltage developed between the two nodes i and j when a unitary electric current flows from i to j. The matrix of all the effective resistances, namely Ω, is computed as Doyle and Snell (1984), Klein and Randić (1993), Van Mieghem (2011)

$$\Omega = \zeta u^T + u\zeta^T - 2L^+ \tag{2}$$

where the vector

$$\zeta = \left(L^+_{11}, L^+_{22}, \ldots, L^+_{NN}\right) \tag{3}$$

contains the diagonal elements of the Moore-Penrose *pseudoinverse* matrix L^+ of the weighted Laplacian matrix $\widetilde{L}$ of G and u is the all one vector. To compute a single element of this matrix, we use the following expression:

$$\omega_{ij} = (e_i - e_j)^T L^+ (e_i - e_j) = l^+_{ii} + l^+_{jj} - 2l^+_{ij} \tag{4}$$

where e_k is the basic vector with the m-th component equal to $(e_k)_m = \delta_{mk}$ and δ_{mk} is the Kronecker-delta: $\delta_{mk} = 1$ if $m = k$, otherwise $\delta_{mk} = 0$.

It has been shown that the intra-community nodes have ω_{ij} values smaller than the inter-community nodes Rubido et al. (2013). The reason is that $\omega_{ij} < 1$ when there are parallel paths between nodes, while $\omega_{ij} > 1$ when there are serial paths as for bridge edges between communities.

3.2 MASS

The *minimum absolute spectral similarity* (MASS) Yan et al. (2018) is a quality index measuring the difference between the spectral properties of a graph and its sparsified version after edge removals. The measure specifically quantifies the difference between the Laplacian L of G and the Laplacian L' of the sparsifier G'. The minimum relative spectral similarity (MRSS) between L' and L is usually computed as:

$$\delta^R_{min} = min_{\forall z} \frac{z^T L' z}{z^T L z} \tag{5}$$

where z can be any vector with N elements and $z^T L' z$ is the Laplacian quadratic form. The vector z intuitively represents

the direction along which the difference between the two graphs is measured. As such, the minimum value of similarity reflects the worst case. However, if G' disconnects into components, the MRSS value becomes zero, making the use of this measure unstable for many optimization algorithms. An alternative viable measure is the *absolute spectral similarity* proposed in Yan et al. (2018):

$$\delta(z) = 1 - \frac{z^T \Delta L z}{z^T \lambda_N z} \tag{6}$$

where λ_N is the largest eigenvalue of L, and $\Delta L = \Delta D - \Delta A$ is the Laplacian of the difference graph ΔG having the same set of nodes of G and the set of edges removed during the sparsification. Since the input vector z is variable, considering the worst-case scenario, the *minimum absolute spectral similarity* (MASS) is

$$\delta_{min} = min_{|z|=1} \left(1 - \frac{z^T \Delta L z}{\lambda_N}\right) = \frac{\lambda_N - \lambda_N^{\Delta}}{\lambda_N} \tag{7}$$

where λ_N^{Δ} is the largest eigenvalue of the difference Laplacian ΔL and, without loss of generality, only the unit length vectors $|z| = 1$ are considered.

The MASS is able to practically quantify the robustness of a network at a mesoscopic (i.e., communities) level when edges are removed and the network disconnects. Ranging between 0 and 1, the MASS offers a practical and computationally efficient similarity measure between the original graph and its version after the edge reduction, indicating whether the spectral properties of the original graph are kept or not after its perturbation. In our works adopting the MASS index, after having analyzed the MASS performance jointly with other network metrics on the set of benchmark networks provided in Yan et al. (2018), we consider 3 categories of similarity: a) high ($\delta_{min} \geq 0.8$), (b) medium ($0.5 \leq \delta_{min} < 0.8$) and c) low ($\delta_{min} < 0.5$).

3.3 Kernels on graphs

Given a weighted undirected graph G with vertices set $V(G) = \{1, ..., N\}$ and *weighted adjacency matrix* $\widetilde{A}$ with elements

$$\widetilde{a_{ij}} = \begin{cases} \text{weight of edge (i,j)} & \text{if i} \neq \text{j} \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

a *kernel on a graph*[2] is a similarity measure between the vertices of G. Informally, given a weighted graph G, a *similarity measure* on the set of vertices $V(G)$ is a bivariate function $k : V(G) \times V(G) \to \mathbb{R}$ quantifying the closeness or affinity between two vertices in a *meaningful* manner Avrachenkov et al. (2019) and thus in a more adequate and functional manner for the targeted application. A kernel on a graph is represented through a symmetric positive semi-definite matrix K, called Gram matrix, with real entries Avrachenkov et al. (2017). More specifically, every kernel is the Gram matrix of some set of vectors in a Euclidean space and the corresponding distance matrix Δ can be obtained as

$$K = -\frac{1}{2} H \Delta H \tag{9}$$

where $H = I - \frac{1}{N} 1 \cdot 1^T$.

Kernels are widely used in data analysis, especially in the field of graph-based machine learning, link prediction, data clustering and community detection. Usually, the choice of a kernel is application-dependent. For graph-based applications, given two nodes i and j connected by an edge weight $\widetilde{a_{ij}}$, the kernels widely used to transform $\widetilde{a_{ij}}$ in a similarity value k_{ij}, include:

- Gaussian k_{ij}=$exp(-\widetilde{a_{ij}}^2/2\sigma^2)$;
- Gaussian RBF k_{ij}=$exp(-\gamma\widetilde{a_{ij}}^2)$;
- Exponential k_{ij}=$exp(-\widetilde{a_{ij}}/2\sigma^2)$.

More in general, following the taxonomy in Avrachenkov et al. (2019), the kernel on graphs can be classified into three main classes: (1) adjacency matrix based, (2) Laplacian based and (3) Markov matrix based. The following subsections describe the most relevant kernels of each class.

3.3.1 Adjacency matrix based kernels

These kernels are based on the weighted adjacency matrix $\widetilde{A}$ of a graph G. Beyond Gaussian, Gaussian RBF and the Exponential which can be considered part of this kernels, this class includes:

- Katz: $K^{Katz}(\alpha) = [I - \alpha\widetilde{A}]^{-}1$, with $0 < \alpha < (\rho(\widetilde{A}))^{-}1$, where $\rho(\widetilde{A}))$ is the spectral radius of $\widetilde{A}$;
- Communicability: $K^{Comm}(t) = exp(t\widetilde{A})$.

3.3.2 Laplacian based kernels

This class relies on the Laplacian L of the graph G including:

- Heat: $K^{Heat}(t) = exp(-tL)$;
- Normalized Heat: $K^{NHeat}(t) = exp(-t\mathcal{L})$ where $\mathcal{L} = \Delta^{-1/2} L \Delta^{-1/2}$ is the normalized Laplacian;

[2] As in Avrachenkov et al. (2019), we prefer to write *kernel on a graph* rather than *graph kernel* since this last term refers to a kernel between graphs. We are interested in measuring the similarity between the nodes of a graph rather than two graphs.

- Regularized Laplacian: $K^{RLap}(t) = [I + tL]^{-1}$.

3.3.3 Markov matrix based kernels

These kernels are applied on the Markov matrix $P = D^{-1}\widetilde{A}$ and include:

- Personalized PageRank: $K^{PPR}(\alpha) = [I + \alpha P]^{-1}$;
- Modified Personalized PageRank: $K^{MPPR}(\alpha) = [I - \alpha D^{-1}\widetilde{A}]^{-1}D^{-1} = [D - \alpha\widetilde{A}]^{-}1$, with $0 < \alpha < 1$;
- PageRank Heat Similarity Measure: $K^{PRHSM}(t) = exp(-t(I - P))$.

4 Method description

In this section, we describe *KOmeGAnet+*, the proposed community detection algorithm. The method uncovers the communities of a graph through two phases. In the pre-processing phase, it converts the input unweighted graph *G* into a sparse effective resistance and kernel based weighted graph G'. In the subsequent optimization phase, the communities are detected through an evolutionary method executed on G'. More specifically, the algorithm weights the input graph edges by first computing the effective resistance value between each couple of nodes and then, it applies a kernel function to obtain a similarity value for the weights. Then, it reduces the input graph *G* to a smaller graph G' through a *weight thresholding* technique that sparsifies the graph eliminating a percentage of edges *nr* with the lowest weights, thus keeping the edges with the highest similarity. In order to quantify how many edges to cut (i.e. *nr*), the algorithm computes the MASS for different fractions of edge removals in order to identify the maximum percentage of removals preserving the community structure. Secondly, similarly to *OmeGAnet*, the method runs a GA on G' maximizing the weighted modularity to detect network communities. In the following, we define the problem we tackle and the steps of the algorithm.

4.1 Problem definition

Given the input graph *G* and its reduced graph G', *find a partition* $C = \{C_1, ..., C_k\}$ *of* G' *in k communities such that the weighted modularity of C is maximized*.

The weighted modularity *Q* is computed as

$$Q = \frac{1}{2m}\sum_{ij}\left(\omega_{ij} - \frac{d_i d_j}{2m}\right)\delta(c_i, c_j) \tag{10}$$

where *m* is the sum of the edges of G', ω_{ij} is the weight of edge (i,j) in G', d_i and d_j are the weighted degrees of nodes *i* and *j* respectively, c_i and c_j are the communities of the corresponding nodes, and δ is the Kronecker function which yields 1 if *i* and *j* are in the same community, that is $c_i = c_j$, zero otherwise. The modularity of a partition basically computes the expected number of edges within the communities of a random graph with the same degree distribution. We apply this objective function to our GA for uncovering communities with both high intra-community similarity (i.e., the overall distance between the nodes of the community is low) and high intra-cluster weighted modularity.

4.2 *KOmeGAnet+* algorithm

KOmeGAnet+ starts creating a population of individuals/chromosomes, where an individual represents a partition in communities, that are initially randomly generated, and then lets the population evolve through variation and selection operators by optimizing the weighted modularity while exploring the search space. Each individual *I* is represented with the *locus-based* adjacency representation Park and Song (1998): *I* is a vector of *n* genes, where each gene is a node and can assume a value in the range {1,...*N*}. A value *z* assigned to a gene *i* means that there is an edge between *i* and *z*. Finally, a decoding step identifies all the connected components of the graph which correspond to the communities found.

KOmeGAnet+ uses the *uniform crossover* which generates a random binary mask of length equal to the number of nodes. The resulting offspring is created by selecting from the first parent the genes where the mask is 0, and from the second parent the genes where the mask is 1. As mutation operator, *KOmeGAnet+* randomly assigns the value of a *i*-th gene to one of its neighbors.

The steps of the algorithm are described by the pseudocode in Algorithm 1. Finally, Figure 1 depicts the pre-processing phase of the method.

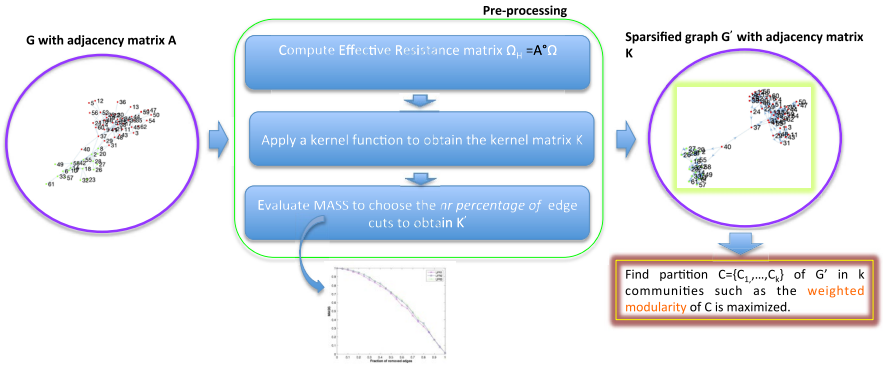

Fig. 1 *KOmeGAnet+* prep-processing steps and subsequent community detection over a sparsified graph

Require: A graph G, kernel type k_{id}, percentage of edges cut nr, maximum number of generations $maxGen$, population Size $popSize$, crossover fraction cf, mutation rate mr.
1: initialize a population of random individuals by assigning to each node one of its neighbors;
2: compute the Laplacian L of G;
3: compute the Moore-Penrose $pseudoinverse$ matrix L^+ of L;
4: compute the effective resistance matrix Ω from L^+ as $\omega_{ij} = l^+_{ii} + l^+_{jj} - 2l^+_{ij}$;
5: compute the kernel matrix K_Ω by applying k_{id} to Ω ;
6: compute the Hadamard product $K_{\Omega H} = A \circ K_\Omega$ between the adjacency matrix A and K_Ω;
7: generate a sparse $similarity\ weighted\ matrix$ as $K'_{\Omega H}$ by eliminating a percentage nr of edges;
8: run the [GA on $K'_{\Omega H}$ for a number of iterations by using modularity as fitness function to maximize, uniform crossover and neighbor mutation as variation operators;
9:
10: **while** termination condition is not satisfied **do**
11: for each individual $I = \{g_1, g_2, ..., g_n\}$ of the population
12: decode I to generate a partitioning
13: evaluate the objective function
14: end for each
15: select the solution having the best fitness value and apply the variation operators to create the next population
16: **end while**
17: return the partition $C = \{C_1, \ldots, C_k\}$ corresponding to the solution with the highest fitness value.

Algorithm 1 The *KOmeGAnet+* method

5 Simulation setup

The proposed method and its contestant methods have been tested on several datasets that will be briefly described in this section. More specifically, here, we describe the real-world and the synthetic networks used, the quality index adopted for testing the behavior of the various algorithms and finally, the set of algorithms that have been simulated and compared on the performed experiments. The simulations have been carried out with Matlab 2020a and the Global Optimization Toolbox.

5.1 Datasets

5.1.1 Real-world networks

We started considering three real-world networks for which the ground-truth division in communities is known.

- **Zachary Karate Club**. This dataset describes the social network between friends of a University Karate Club, collected and studied by Wayne Zachary in 1977. 34 members of the club were studied for a period of three years and the networks describe the interactions had between pairs of members who interacted outside the club. The group splitted into two communities almost of the same size due to disagreements between the members.
- **American College football**. The dataset contains the network of American football games between Division IA colleges during regular season Fall 2000. The graph is composed by 115 nodes that are the American football teams, connected by 616 edges representing the games between two teams. The overall number of communities is 12.
- **Bottlenose dolphins**. This network contains the associations between 62 Bottlenose dolphins observed by David Lusseau, a researcher at the University of Aberdeen. These dolphins were located in Doubtful Sound, a

Table 1 LFR-128 parameters setting

Parameter	Value
Number of nodes	128
Node average degree (*d*)	8
Node maximal degree (*maxd*)	9
Exponent for power law creating degree sequence	2
Exponent for power law creating community sizes	1
Mixing parameter μ	[0.1; 0.6]
Maximal community size (*minc*)	40
Minimal community size (*maxc*)	20
Average density	0.02

Table 2 LFR-1000 parameters setting

Parameter	Value
Number of nodes	1000
Node average degree (*d*)	[60, 80, 250, 500]
Node maximal degree (*maxd*)	[70, 90, 300, 600]
Exponent for power law creating degree sequence	2
Exponent for power law creating community sizes	1
Mixing parameter μ	[0.1; 0.6]
Maximal community size (*minc*)	400
Minimal community size (*maxc*)	100

very large and naturally imposing fiord in Fiordland, in the far south west of New Zealand. The graph contains 159 edges and the ground-truth has two classes.

5.1.2 Synthetic networks

We created a pool of synthetic networks with realistic community structures using the Lancichinetti-Fortunato-Radicchi (LFR) benchmark Lancichinetti et al. (2008), an algorithm that generates artificial networks that resemble real-world networks with node degrees and community sizes distributed according to a power law. By setting the *mixing parameter* μ, the user can control the fraction of edges between communities. The lower the μ, the lower the inter-community links. On the contrary, when μ has high values, the community structure is more random, with many inter-community links. At the extremes, when $\mu = 0$ all the edges are intra-community links of a unique community, while if $\mu = 1$ all the edges are between nodes belonging to different communities.

The parameters used for generating the LFR networks are shown in Table 1 and in Table 2, for the networks with 128 and 1000 nodes, respectively. In particular, for each μ value we generated 10 network instances.

5.2 Evaluation measure

To assess the quality of the partitions found by the community detection, the *Normalized Mutual Information (NMI)* is used. Given two partitionings of a network X and Y, and C the confusion matrix whose element C_{ij} is the number of nodes of community i of the community division X that are also in the community j of the community division Y, the NMI between the two partitionings is computed as:

$$NMI(X,Y) = \frac{-2\sum_{i=1}^{c_X}\sum_{j=1}^{c_Y} C_{ij} log(C_{ij} n / C_{i.} C_{.j})}{\sum_{i=1}^{c_X} C_{i.} log(C_{i.}/N) + \sum_{j=1}^{c_Y} C_{.j} log(C_{.j}/N)} \quad (11)$$

where c_X (c_Y) is the number of communities in X (Y), $C_{i.}$ ($C_{.j}$) is the sum of the elements of C in row i (column j), and N is the number of nodes. If $X = Y$, $NMI(X,Y) = 1$, while $NMI(X,Y) = 0$ if the two partitionings are completely different.

5.3 Algorithms in comparison

It follows a brief description of the contestant methods.

- *Louvain* Blondel et al. (2008): this method is based on a greedy modularity optimization approach. First, the algorithm identifies small communities by locally optimizing modularity. Then, it builds a new network whose nodes are the communities previously found, and these steps are repeated until a hierarchy of high-modularity communities is obtained.
- *Infomap* Rosvall and Bergstrom (2008): this method exploits the principles of information theory by defining the community detection problem as the problem of finding a description of minimum information of a random walk on the graph. The method maximizes the Minimum Description Length as objective function by quickly providing an approximation of the optimal solution.
- *Spectral modularity* Newman (2013): this is a spectral method that uses the leading eigenvector of the *modularity matrix B* associated to G to recursively split the communities of the network into two until no further improvement of modularity is obtained. At each step the modularity contribution ΔQ in a possible subdivision of the network is computed: if the ΔQ value is not positive the algorithm does not subdivide any community.
- *ERGA*: this is a baseline GA implemented here and running on the input matrix weighting the edges with the effective resistance and optimizing Newman modularity.
- *ZB* Zhang and Bu (2019): Zhang and Bu's algorithm first compute the similarity of each pair of nodes in G using a Gaussian function of the resistance distance and convert the original network in a weighted network G'. Then, it applies a repeated bisection spectral method using the normalized Laplacian matrix of G'. Differently

from our approach, for the resistance distance it is not considered its square root.

- *OmeGANet* Pizzuti and Socievole (2021a): a GA exploiting effective resistance for weighting the edges and applying a sparsification procedure that keeps a given number of *nn* nearest neighbors having the lowest values of effective resistance (i.e. connections with high similarity).
- $OmeGANet+$ Socievole and Pizzuti (2023b): *OmeGANet* improvement removing edges based on the minimum absolute spectral similarity (MASS) and selecting the sparsification level that maximizes MASS while preserving graph connectivity.
- *ERKGA*: a version of *ERGA* implemented here and running over a kernel-based matrix, where the similarity value is a kernel of the effective resistance. This GA-based algorithm adopts the same locus-based representation, initialization, crossover and mutation operators of *ERGA*.
- *KOmeGANet*: kernel-based version of *OmeGANet* implemented here.
- *LPA* Kim et al. (2022): this method prunes links according to node similarity (Jaccard's Index, number of common triangles, Forman-Ricci Curvature) setting a pruning threshold α, then runs Louvain to detect the communities. This sparsification strategy makes a trade-off between efficiency (running time) and accuracy (modularity, conductance).
- *KLPA*: kernel-based version of *LPA* implemented here.

6 Results

The simulations have been organized into three different experiments. Before evaluating *ERKGA*, we performed extensive simulations to assess the impact of kernels on the community detection performance. In a first experiment, the focus has been *testing the effectiveness of kernels and finding the best performing kernels* on an effective resistance based community detection scheme. Since the proposal is based on a GA running over a kernel-based matrix of the effective resistance matrix, in the Experiment 1, we evaluated *ERKGA*, which is basically *KOmeGAnet+* without the sparsification. Note that we started performing this experiment in order to better understand the impact of kernels, isolating this aspect from the sparsification effect. This analysis comes first *KOmeGAnet+* simulations in order to better highlight the relationship between kernels and effective resistance based community detection based on a GA. Once assessed the usefulness of kernels over *ERKGA*, in the Experiment 2, we compared *ERKGA* to other schemes by running a *comparison between kernel-based and non kernel-based algorithms*. Finally, in the Experiment 3, the results of the *KOmeGAnet+ evaluation* are reported.

6.1 Experiment 1: comparison between different kernels in terms of community detection performance

We have initially studied how the *ERKGA* NMI varies for each kernel, when real-world networks are considered. The results of the simulations are shown in Figures 2, 3, 4 and 5, where the kernels have been grouped into exponential, adjacency-based, Laplacian-based and Markov matrix based, respectively. As genetic parameters we have set maximum number of generations 100, population size 200, crossover fraction 0.9 and mutation rate 0.1. Each NMI value has been averaged over 10 runs of the method. Figure 2 illustrates the variation of NMI values for exponential kernels (Gaussian, RBF, and Exponential) applied to three real-world networks: Karate, Football, and Dolphins. The results show that NMI values vary depending on the kernel parameter. For example, the Gaussian kernel achieves the highest NMI (0.8589) for the Karate network when $\sigma = 0.1$, while performance declines for higher σ values. This trend highlights the sensitivity of Exponential kernels to parameter tuning. Figure 3 shows the performance of adjacency-based kernels (Katz and Communicability) on the same real-world networks. The Katz kernel performs exceptionally well, achieving the highest NMI for the Football network (0.9095). The Communicability kernel also provides competitive results, but its performance varies more significantly with parameter changes. In Figure 4, Laplacian-based kernels (Heat, Normalized Heat, and Regularized Laplacian) are shown. These kernels exhibit moderate performance across the datasets, with NMI values peaking for specific parameter values. For example, the Heat kernel achieves good results for the Karate network but struggles on the Football and Dolphins networks. Finally, Figure 5 presents the results for Markov matrix-based kernels (Personalized PageRank, Modified Personalized PageRank, and PageRank Heat Similarity Measure). These kernels generally perform worse than adjacency-based kernels, with lower NMI values across all networks. The Personalized PageRank kernel achieves the best results among this group, particularly for the Dolphins network.

To establish which kernel is the most suitable for these real-world networks, we compared the highest NMI value achieved by *ERKGA* for each applied kernel (Table 3). For the Katz kernel, α_{max} is equal to 0.393, 0.5034, and 0.4502 for Karate, Football and Dolphins networks, respectively. As expected, it is worth pointing out that the best kernels for the real-world datasets considered are the ones based on the adjacency matrix: the Gaussian kernel for the Zachary

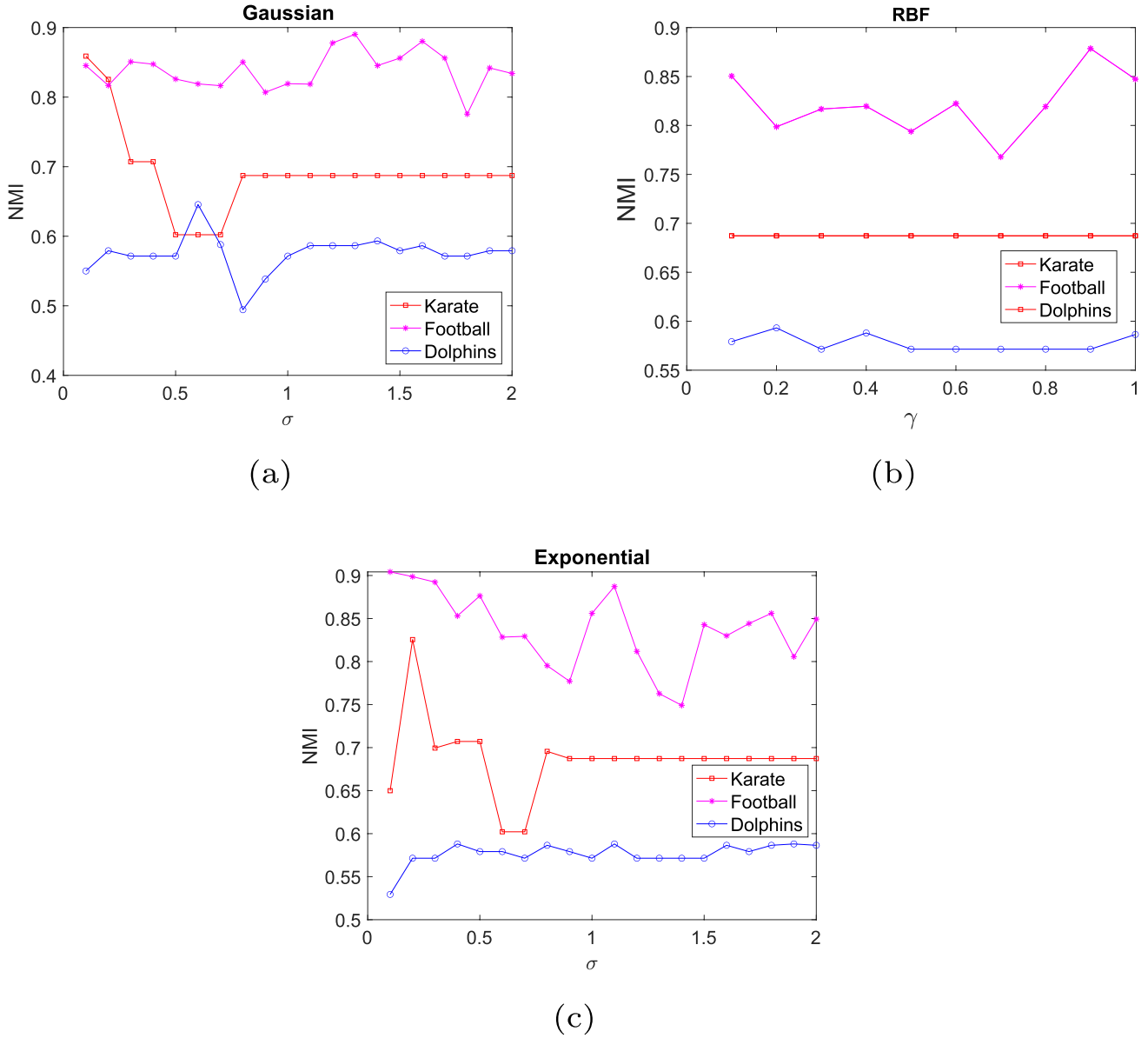

Fig. 2 Variation of NMI value for the exponential kernels over the real-world networks

Karate Club (0.8589) and the Bottlenose Dolphins (0.6455) networks, and the Katz kernel for the American College Football network (0.9095). We previously underlined that those types of kernels are commonly considered the most suitable for graph-based applications.

We have also tested synthetic networks with 128 nodes (Figures 6, 7, 8 and 9), generated with mixing parameters varying between 0.1 and 0.6. In Table 4, we report the values of spectral radius used for the Katz kernel: as a consequence, we have set $\alpha = 0.5$. As genetic parameters, we have set maximum number of generations 100, population size 700, crossover fraction 0.9 and mutation rate 0.1. Again, each run has been averaged over 10 runs of the method. For $\mu = 0.1$, the choice of a kernel is not influential, every kernel function achieves NMI 1. In this case, the mixing parameter generates dense communities with few inter-links between them. As a result, the role of the different kernels is equivalent since the few inter-community edges have an effective resistance value significantly different from the intra-community links values: this allows the algorithm to better distinguish the edge weights.

For $\mu = 0.2$, the Gaussian kernel (Figure 2a) perfectly matches the ground-truth for many σ values. For $\mu = 0.3$, the maximum NMI value is achieved by the exponential kernel at $\sigma = 0.9$ (Figure 6a). For $\mu = 0.4$ and $t = 0.8$, the Heat achieves 0.3791 which is the maximum NMI value for this mixing parameter over all the considered kernels (Figure 4a). For $\mu = 0.5$ and $t = 1.7$, the best performer is the

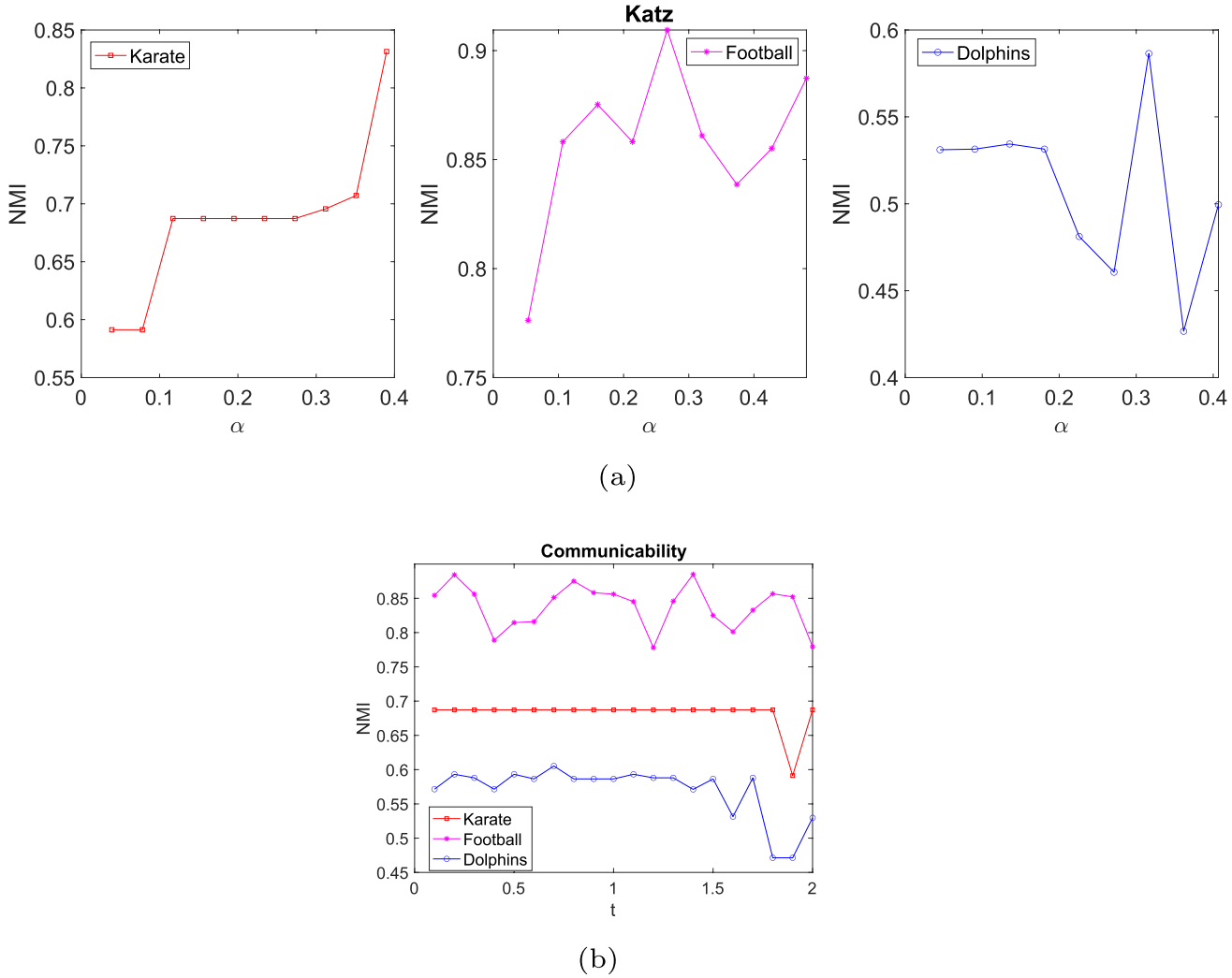

Fig. 3 Variation of NMI value for the adjacency-based kernels over the real-world networks

PageRank Heat Similarity Measure achieving 0.2123 (Figure 5c). Finally, for $\mu = 0.6$, with Regularized Laplacian and $t = 1.5$, the algorithm achieves 0.2005 (Figure 4c). Overall, we can conclude that the kernels based on the adjacency matrix of the graph (Gaussian, Exponential, Katz) work well on communities with low mixing parameter ($\mu = 0.2$ and $\mu = 0.3$, where the networks are well structured into groups. On the contrary, when the network structures are more randomly organized ($\mu = 0.4$, $\mu = 0.5$ and $\mu = 0.6$), Laplacian or Markov matrix based kernels are needed.

6.2 Experiment 2: comparison between different kernel-based and non kernel-based community detection algorithms

Once assessed which kernel is the most suitable for some benchmark networks, we made a comparison between different algorithms. The aim of this second experiment was two-fold: compare kernel-based and non kernel-based schemes, and in particular, evaluate the role of the kernels over methods based on GAs since our proposal is based on this evolutionary method.

Real-world networks. The results in Tables 5, 6 and 7 show a comparison between different community detection benchmarks over the real-world networks. The first row shows the ground truth, i.e. the real underlying community structure in terms of number of communities and modularity. Then, we report the results of three benchmark algorithms that usually are best-performing without the use of kernels (*Louvain*, *Infomap*, *Spectral modularity*), the results of a kernel-based method not based on GAs (*ZB*), and finally, three GA-based algorithms: *ERGA*, *OmeGAnet*, and *ERKGA* which uses kernels[3]. For *OmeGAnet*, we consid-

[3] The results of ZB and Spectral modularity have been taken from the reference Zhang and Bu (2019).

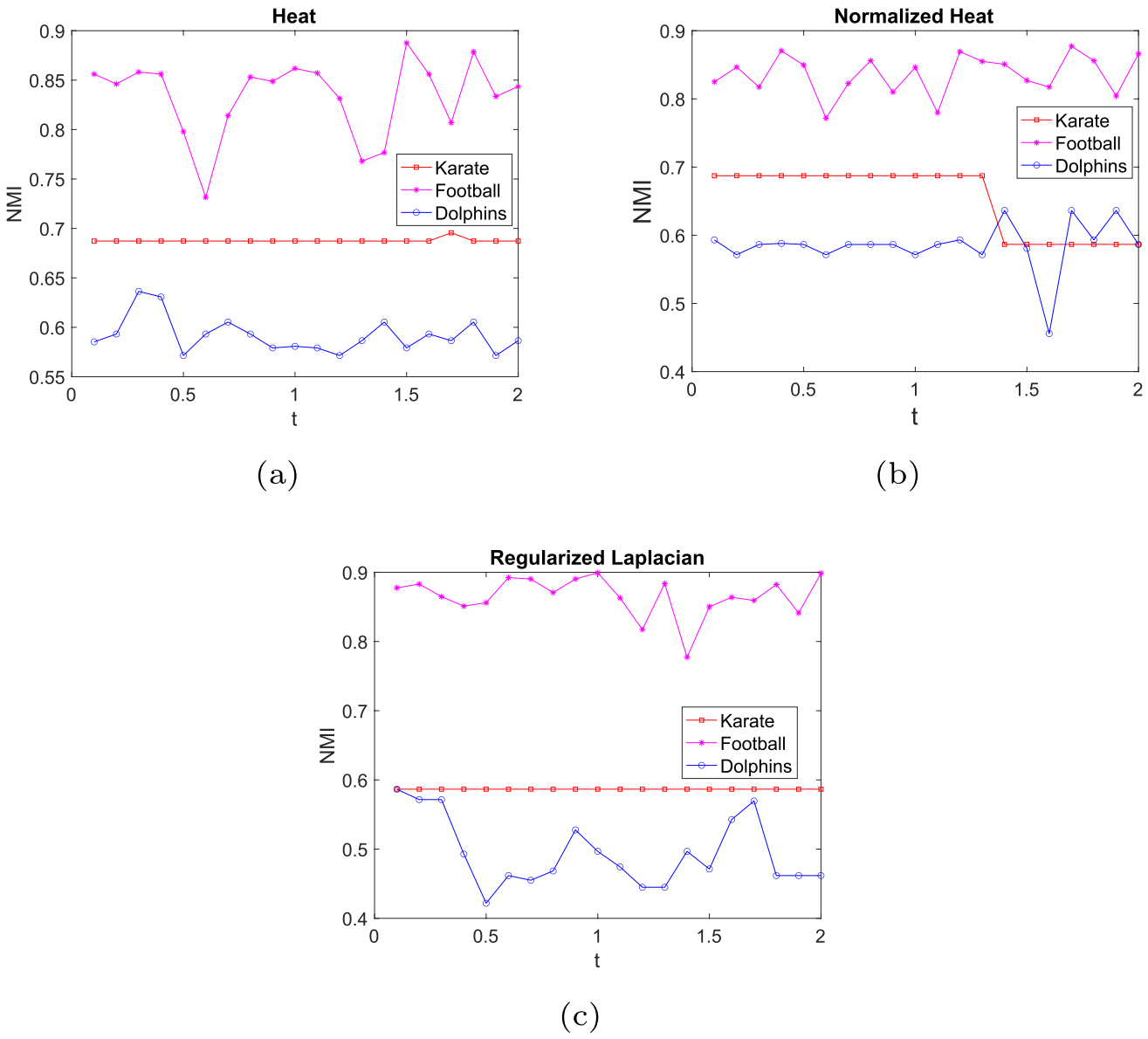

Fig. 4 Variation of NMI value for the Laplacian-based kernels over the real-world networks

ered $nn = 4$ as sparsification threshold, while for *ERKGA* we applied the Exponential kernel with $\sigma = 0.1$.

Over the Zachary Karate Club network, the GA-based algorithms based on the effective resistance perform the best. *ERKGA* results in the highest NMI (0.8959), followed by *ERGA*. Hence, both the effective resistance weighting the graph and the kernels are necessary. *OmeGAnet* and *Infomap* only achieve 0.6995. The *spectral modularity* method shows how the repeated subdivision of a network into two parts will in some cases fail to find the optimal optimum: the communities found are 4 while it would be sufficient to run a single bipartitioning step (if running a supervised version of the algorithm) to match the true division. Surprisingly, the *ZB* method that uses a kernel function of the effective resistance to weight the graph achieves only 0.5866: similarly to the *spectral modularity* method, the bipartitioning is not effective over this dataset, if compared to a GA-based logic. Finally, it is worth noting the poor performance of the benchmark *Louvain* that achieves only 0.5195.

Over the American College Football network, *Louvain* and *Infomap* perform the best with an NMI of 0.9269 and 0.9242, respectively. *OmeGAnet* and *ERKGA* achieve similar results with 0.9151 and 0.9095, respectively. In this case the sparsification strategy realized by *OmeGAnet* allows to better uncover the communities. Concerning the kernel application, it is worth noting that for a GA-based algorithm is necessary over this dataset: *ERGA* only achieves 0.341 with a number of communities that is completely different

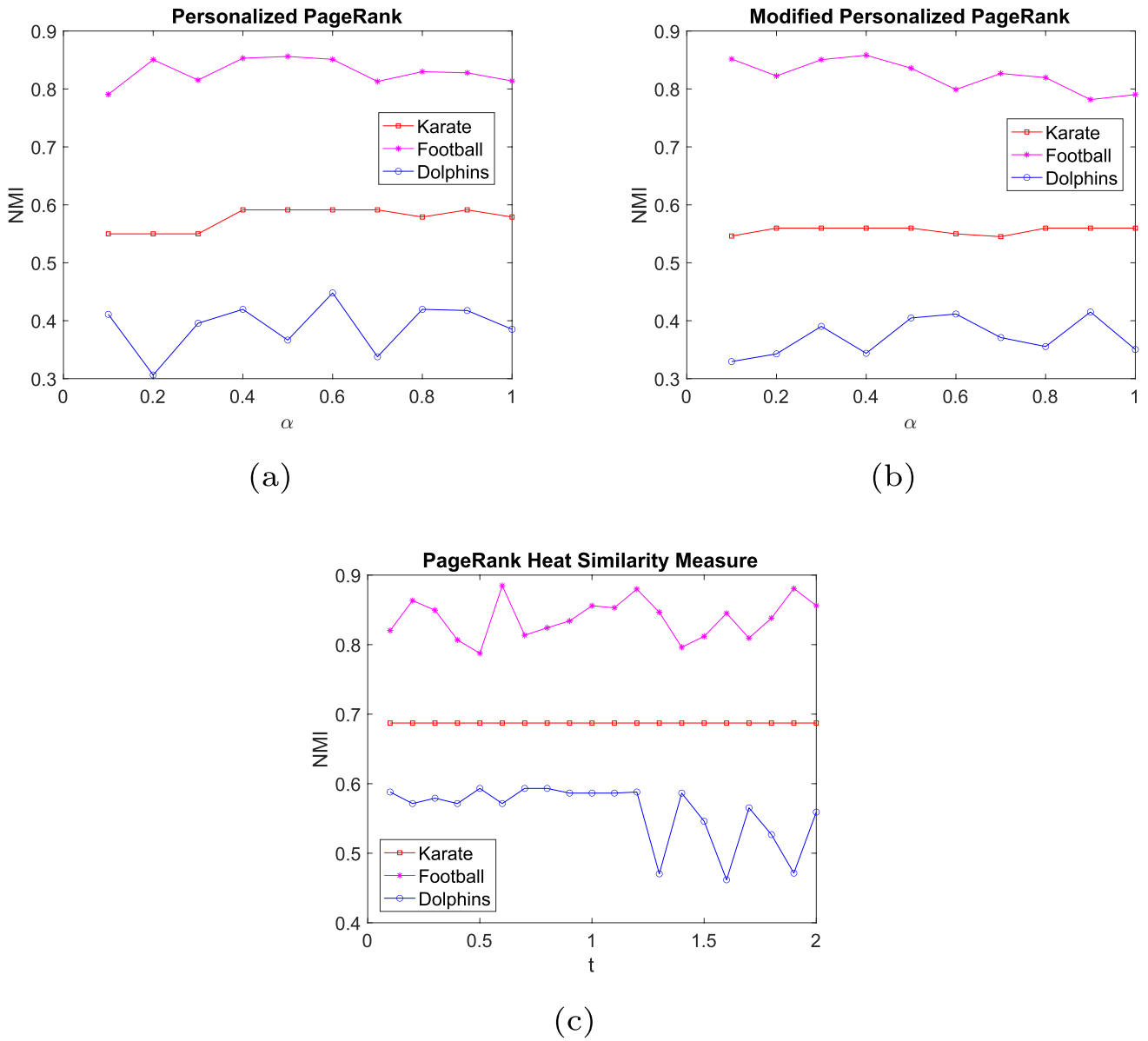

Fig. 5 Variation of NMI value for the Markov matrix based kernels over the real-world networks

from the ground truth.Finally, diffrently form the Zachary Karate Club, here the multi-community division based on the spectral modularity implemented by *ZB* works better.

The results of the Dolphins networks confirm the good performance of the GA-based algorithms in conjunction with effective resistance and kernels. *ERKGA* performs the best with an NMI of 0.6455, followed by *OmeGAnet* with a value of 0.5792. *ERGA*, on the contrary, only achieves 0.5138 demonstrating again that the kernels are able to improve its performance. Overall, the GA and kernel-based algorithms, outperform the non kernel-based schemes *Louvain*, *Infomap* and the *spectral modularity*.

LFR-128 networks. Tables 8, 9 and 10 show the results of the comparison on the synthetic networks with 128 nodes. Here, for *OmeGAnet*, we set $nn = 5$. Differently from the real networks, these graphs are denser and with much more inter-community links as the mixing parameter μ increases. Here, we observed that the kernels are needed when the effective resistance is used since the edge weights have very similar values. The NMI performance obtained by *ERGA* is indeed poor and this behavior is even more notable for the networks with low mixing parameter where there are greater number of inter-community links. Intuitively, the inter-community links, when are few, have an effective resistance value that is higher than the intra-community links where the distance between nodes is lower. Each node of a community can usually reach the other nodes within its community"more easily". On the contrary, when many

links span across two communities (i.e. high μ) the effective resistance value of the inter-community edges may be very similar to the one of the intra-community edges avoiding a proper classification of the nodes into communities.

Overall, *ERKGA* performs the best. For μ values of 0.1 and 0.2, *ERKGA*, *OmeGAnet* and *Infomap* perfectly match the ground-truth, then, for greater μ, *ERKGA* strategy is the most effective. It is worth noting that *OmeGAnet* uncovers the right number of communities at each mixing parameter. Here, we argue that the sparsification procedure helps in avoiding overestimating or underestimating *k* since this procedure cuts the edges that are noisy in terms of effective resistance.

LFR-1000 networks. In another experiment, we made the comparison on larger synthetic networks with 1000 nodes. Here, for *OmeGAnet*, we set $nn = 50$. Figure 10 plots an instance of LFR-1000 network with average edge density 0.06 for each mixing parameter. For $\mu = 0.1$, the network is subdivided into 5 communities: (1) red, (2) green, (3) pink, (4) yellow and (5) blue. For all the other mixing parameters, the ground-truth is composed by 4 communities. One can observe how the community structure tends to converge to a unique big and dense group as the mixing parameter increases since the inter-community links increase. Moreover, even in the $\mu = 0.1$ case, the number of edges across two communities is not low: for the effective resistance based schemes, this challenges the distinction between intra-community end inter-community edges. In this case, the kernels are determinant. Table 11 shows the NMI values of the different community detection strategies. Differently from the previous results over smaller networks with 128 nodes, when running a GA with a network weighted with the effective resistance, the algorithm performance of *ERGA* is terribly poor. On the contrary, when we apply the kernel functions to the effective resistance matrix Ω, the NMI value significantly increases. As an example, for the networks with $\mu = 0.1$, *ERGA* only achieves 0.099, while *ERKGA* 0.7813. *ERKGA* finds 7 communities with 253, 198, 192, 178, 174, 3 and 2 nodes, respectively (see Table 13), while the ground-truth is composed by 5 communities with 251, 206, 190, 178 and 175 nodes, respectively. As such, *ERKGA* fragments some large communities creating two very small groups with 3 and 2 nodes, for this reason it is not able to achieve 1. On the contrary, *OmeGAnet*, which uses the sparsification, is able to achieve the maximum NMI value. Also *Louvain* and *Infomap* find the exact classes.

For $\mu = 0.2$, *ERGA* only achieves 0.019, while *ERKGA* 0.3657. In particular, *ERKGA* finds 7 communities with 450, 321, 205, 15, 4, 3 and 2 nodes, respectively. The ground-truth is composed by 4 communities with 299, 263, 261, and 177 nodes. Once again, *ERKGA* fragments creating very small

Table 3 Kernel comparison across the real-world networks: Zachary Karate Club, American College Football, and Bottlenose Dolphins (k = number of communities)

	Network	Ground Truth	$\mathbf{K}^{Gauss}$	$\mathbf{K}^{RBF}$	$\mathbf{K}^{EXP}$	$\mathbf{K}^{Katz}$	$\mathbf{K}^{Comm}$	$\mathbf{K}^{Heat}$	$\mathbf{K}^{NHeat}$	$\mathbf{K}^{RLap}$	$\mathbf{K}^{PPR}$	$\mathbf{K}^{MPPR}$	$\mathbf{K}^{MSM}$
k	Karate	2	3	4	3	3	4	4	4	4	4	6	4
NMI		1	**0.8589**	0.6873	0.8255	0.8314	0.6873	0.6956	0.6956	0.5866	0.5913	0.5598	0.6873
Q		0.3715	0.4744	0.4198	0.3991	0.3875	0.4198	0.4174	0.4174	0.4188	0.388	0.3892	0.4188
k	Football	12	11	10	11	11	10	10	10	11	8	8	10
NMI		1	0.8903	0.8785	0.9042	**0.9095**	0.885	0.8876	0.8773	0.8993	0.8561	0.8582	0.8848
Q		0.601	0.6046	0.6024	0.603	0.6032	0.6011	0.5978	0.6005	0.5998	0.6044	0.6043	0.6009
k	Dolphins	2	4	5	5	5	5	4	4	5	8	8	5
NMI		1	**0.6455**	0.5932	0.588	0.5865	0.6053	0.6363	0.6363	0.5866	0.448	0.4158	0.5932
Q		0.373	0.5258	0.5277	0.5276	0.5285	0.5259	0.5263	0.5263	0.5264	0.48	0.4163	0.5277

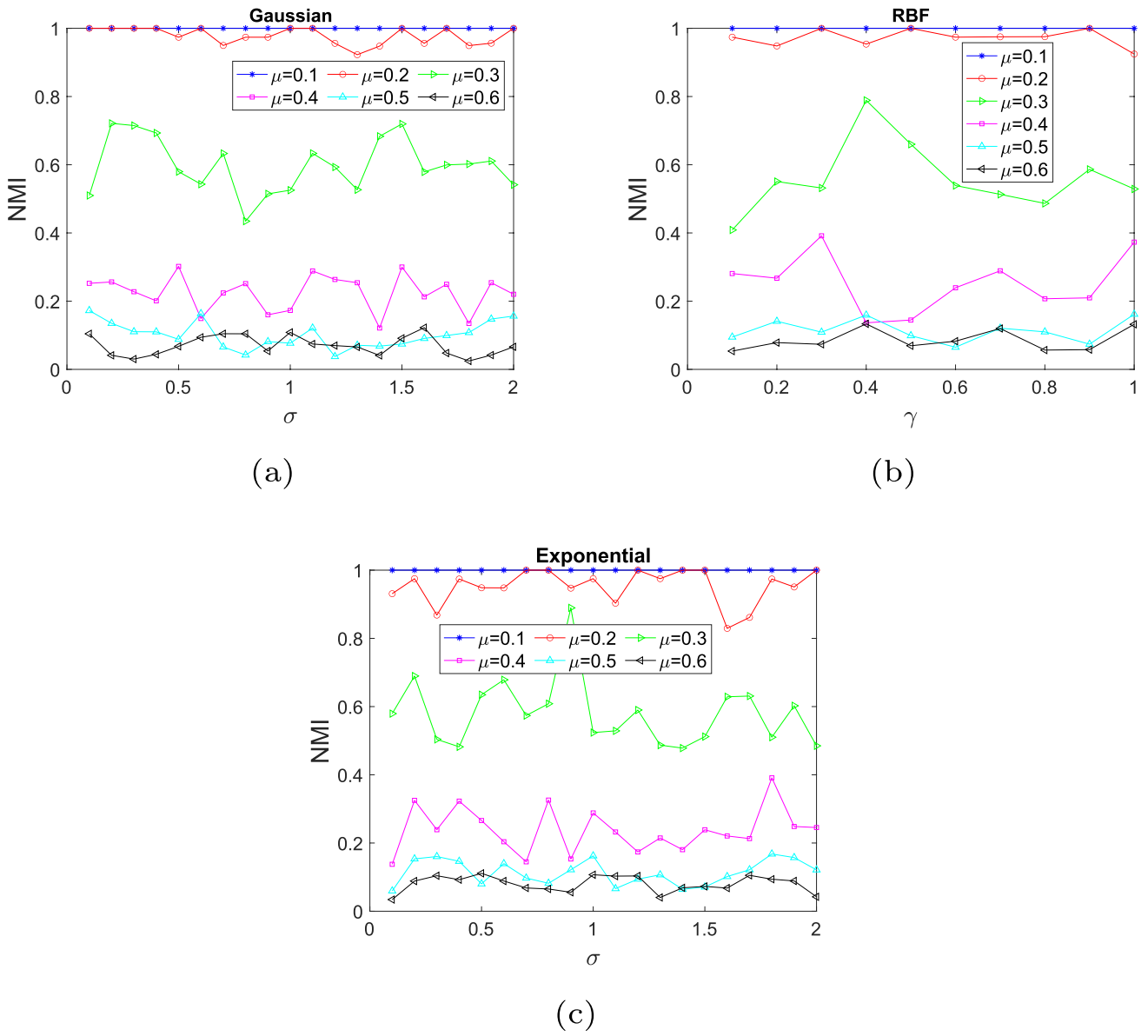

Fig. 6 Variation of NMI value for the exponential kernels over 128-nodes synthetic networks

communities, while *OmeGAnet* is able to cut the noisy links through sparsificiation achieving an NMI value of 0.9867. A similar behavior is observable also over the other mixing parameters. We thus conclude that the kernels are able to improve the community detection performance but when the network has many edges not only within the communities but also between them, we would need a sort edge pruning strategy, as the one implemented by *OmeGAnet*, able to cut the noisy links and allowing a better identification of the ground-truth.

We finally remark that *Louvain* is the best performing, achieving NMI 1 until $\mu = 0.5$ and NMI 0.4779 for $\mu = 0.6$, while *Infomap* puts all the nodes into one community for $\mu = 0.5$ and $\mu = 0.6$ thus resulting in NMI 0.

In Table 14, we compare the algorithms over LFR-1000 networks with different average densities. For the *ERKGA*, the Exponential kernel with $\sigma = 0.1$ was used. This experiment was carried out to further assess the impact of kernels and the sparsification strategy. We considered three networks:

- LFR1, a sparse network with 40071 edges and edge density 0.08, generated with $d = 80$, $maxd = 80$, $minc = 200$ and $maxc = 400$;
- LFR2, a network with 124942 links and density 0.25, generated with $d = 250$, $maxd = 900$, $minc = 200$ and $maxc = 400$;

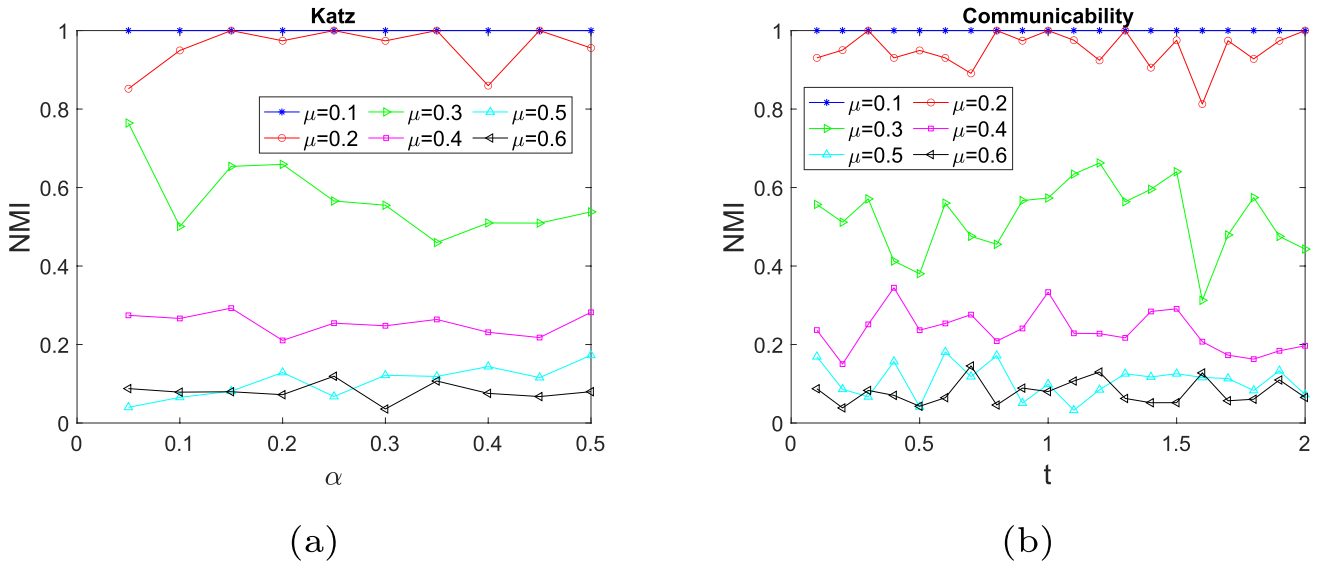

Fig. 7 Variation of NMI value for the adjacency-based kernels over 128-nodes synthetic networks

- LFR3, a dense network with 244795 edges and density 0.49, generated with $d = 500$, $maxd = 600$, $minc = 100$ and $maxc = 200$.

Figures 11, 12 and 13 show their topologies and subdivisions into communities. LFR1 nodes are divided into 3 communities: (1) the blue, (2) the red and the (3) the green community. Similarly, the LFR2 network has 3 communities but is characterized by a greater number of inter-community edges. The LFR3 network is even denser with 559 nodes within a community and 441 nodes into another. For *OmeGAnet*, here we set the following sparsification thresholds: $nn = \{50, 150, 300\}$ for LFR1, LFR2 and LFR3, respectively.

Table 14 shows an interesting insight that confirms our intuition behind *KOmeGAnet+* proposal: the kernels play an important role when the network is weighted with the effective resistance (see *ERKGA* vs. *ERGA*) but the sparsification is needed as well when networks are dense as the results provided by *OmeGAnet* suggest. For the LFR3 network, *OmeGAnet* achieves only 0.3522 as NMI value, while *Louvain* and *Infomap* are able to perfectly match the ground-truth.

6.3 Experiment 3: *KOmeGAnet+* evaluation

We now show what happens when using kernels in conjunction with sparsification. Specifically, we evaluate the performance of *KOmeGAnet+* over networks with different edge densities. For this experiment, we made a comparison between three different kind of sparsification procedures: the one provided by *OmeGAnet* which cuts a percentage of neighbor links pre-defined by the user as input parameter, the *LPA* algorithm sparsification procedure cutting an α threshold of nodes set by the user and based on edge attribute (i.e. the similarity between end nodes), and the one provided by *OmeGAnet+* which uses the MASS as index for deciding the fraction of edge cuts. Table 15 compares *OmeGAnet*, its kernel-based version *K OmeGAnet*, *LPA*, its kernel version *KLPA*, *OmeGAnet+* and its kernel version *KOmeGAnet+* over a pool of LFR networks with 1000 nodes, average density 0.06 and mixing parameters varying between 0.1 and 0.6. For all the kernel-based algorithms, the Exponential kernel has been applied.

As sparsification thresholds, for *OmeGAnet* and *K OmeGAnet*, we set the same number of nearest neighbors *nn* used in the previous experiment. Finally, for *OmeGAnet+* and *KOmeGAnet+*, we computed the MASS during the preprocessing phase of the algorithms, in order to set the *nr* parameter. Figure 14 shows the trend of the minimum absolute spectral similarity for fraction of removed edges ranging from 0.1 to 1. It is worth noting that the spectral properties of the sparsifier are kept high ($MASS \geq 0.8$) until the cut of the 35% of nodes, for all the mixing parameters. Moreover, for a value of 0.35 of fraction of removed edges, the networks remain connected. As such, we select this fraction of cuts for all the networks and for all the algorithms in Table 15. More specifically, for *LPA* and *KLPA* we thus choose $\alpha = 0.35$ and Jaccard similarity as edge attribute. The comparison of Table 15 shows two important aspects: (1) again, kernels improve the NMI in all the algorithms at each μ, (2) *OmeGAnet+* sparsification produces higher NMI gains, especially as the mixing parameter increases, hence, it is preferable. But more specifically, the joint use of this sparsification with kernels (*KOmeGAnet+*) is able to significantly increase the NMI achieving NMI=1 until $\mu = 0.5$.

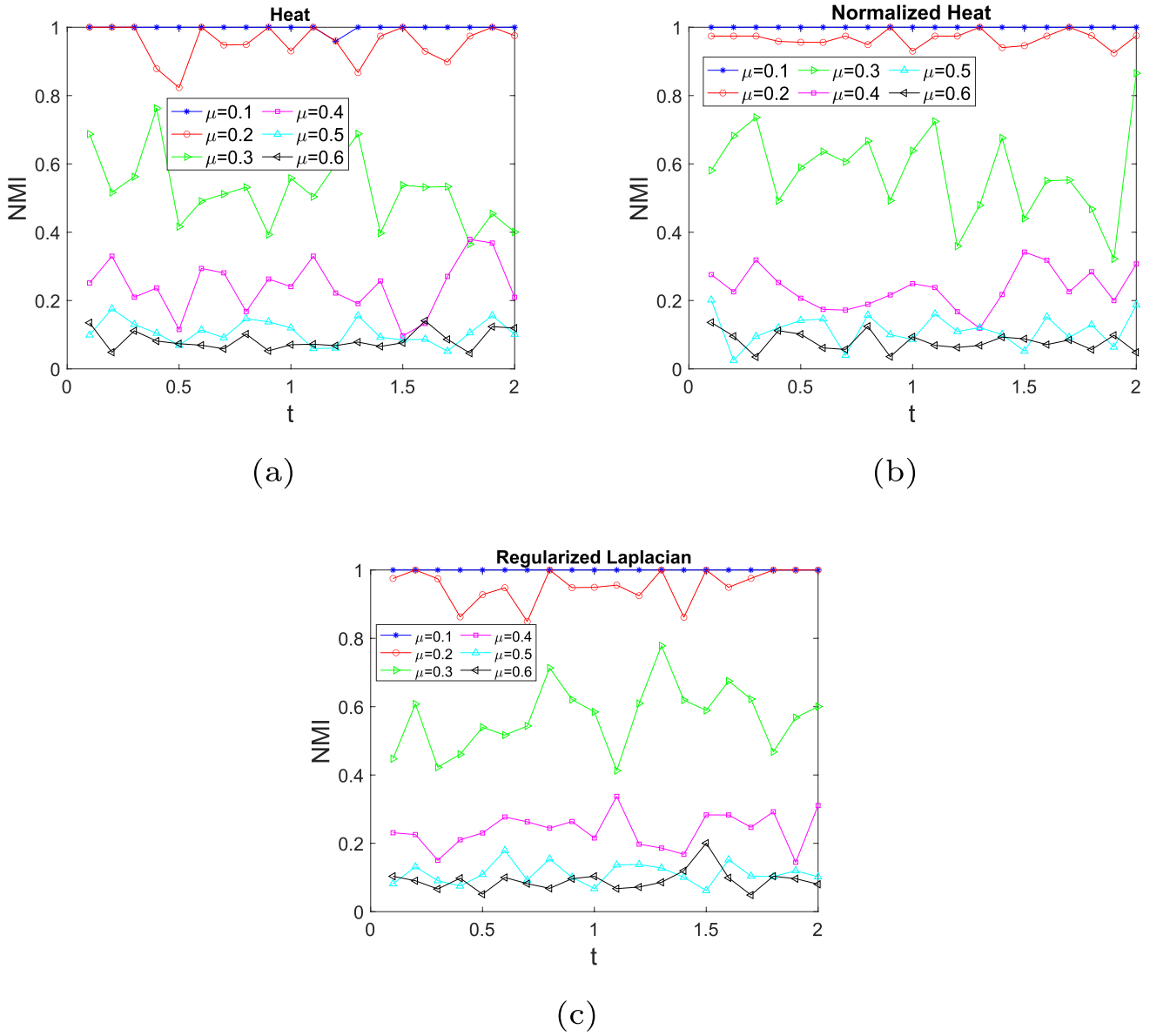

Fig. 8 Variation of NMI value for the Laplacian-based kernels over 128-nodes synthetic networks

Table 16 shows the results of the comparison for the 1000 nodes networks with different edge densities, while Figure 15 the MASS computation for the three networks. Looking at the MASS, we chose to eliminate the 40% of nodes over all the networks since it results in the highest MASS maintaining LFR1, LFR2 and LFR3 networks connected. Again, *KOmeGAnet+* shows an edge over the competition since it is able to achieve NMI=1 even on the LFR3 network with the highest density.

7 Conclusion and further work

This work presented *KOmeGAnet+*, an evolutionary community detection algorithm extending our previous work *OmeGAnet*Socievole and Pizzuti (2023b) which exploits kernel similarity based on the effective resistance, a distance measure derived from the electrical circuits theory, and weight thresholding sparsification to group nodes. The method is based on a pre-processing phase where the edges of the graph are weighted through the effective resistance distance between them, then a kernel-based similarity is computed and finally, weight thresholding sparsification based on the minimum absolute spectral similarity (MASS) is applied to produce a weighted sparsified graph on which

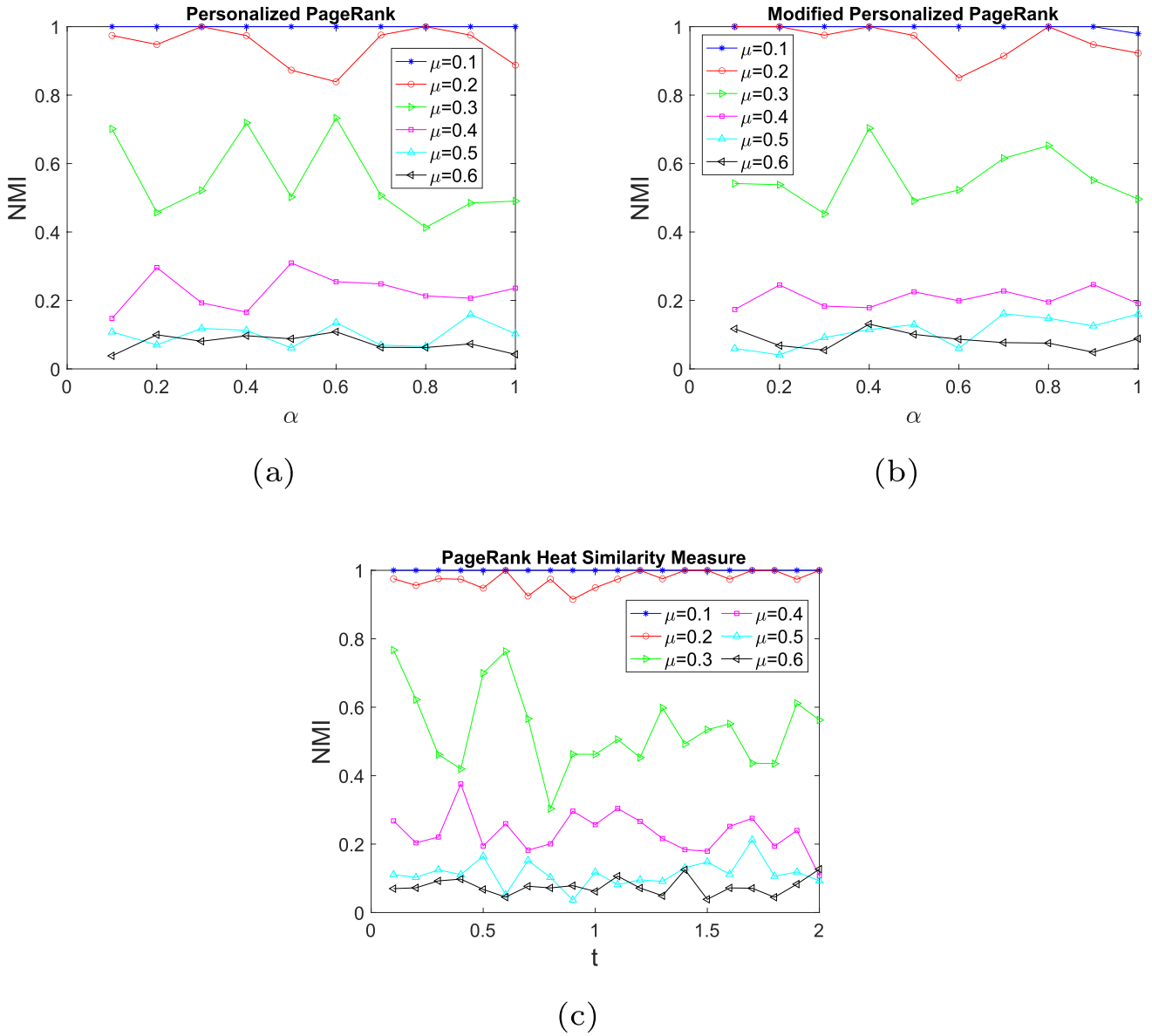

Fig. 9 Variation of NMI value for the Markov matrix based kernels over 128-nodes synthetic networks

Table 4 Values of the inverse of the spectral radius for the LFR-128 networks

	$\mu = 0.1$	$\mu = 0.2$	$\mu = 0.3$	$\mu = 0.4$	$\mu = 0.5$	$\mu = 0.6$
$\rho(\widetilde{A})^{-1}$	0.5019	0.5021	0.5022	0.5022	0.5023	0.5023

Table 5 Algorithms comparison for the Zachary Karate Club network

	k	NMI	Q
Ground-truth	2	1	0.3715
Louvain	3	0.5195	0.402
Infomap	3	0.6995	0.402
Spectral modularity	4	0.6711	0.3934
ZB	4	0.5866	0.4188
ERGA	2	0.8372	0.3718
OmeGANet	3	0.6995	0.402
ERKGA	4	0.8589	0.3744

Table 6 Algorithms comparison for the American College football network

	k	NMI	Q
Ground-truth	12	1	0.6010
Louvain	12	0.9269	0.601
Infomap	12	0.9242	0.6005
Spectral modularity	12	0.7582	0.4936
ZB	8	0.8563	0.5734
ERGA	2	0.341	0.3678
OmeGANet	11	0.9151	0.6008
ERKGA	11	0.9095	0.6032

Table 7 Algorithms comparison for the Dolphins network

	k	NMI	Q
Ground-truth	2	1	0.373
Louvain	10	0.4506	0.4592
Infomap	6	0.5197	0.5146
Spectral modularity	5	0.4489	0.4912
ZB	5	0.5621	0.4934
ERGA	2	0.5138	0.36
OmeGANet	4	0.5792	0.5265
ERKGA	4	0.6455	0.5258

synthetic networks generated with low mixing parameter (i.e., the networks are well structured into groups). On the contrary, when the network structures are more randomly organized, Laplacian or Markov matrix based kernels are needed. In a second experiment, by comparing kernel-based and non kernel-based community detection schemes, we found that the algorithm relying on GAs and kernels are able to outperform the non kernel-based benchmark schemes *Louvain*, *Infomap* and the *spectral modularity*. In

Table 10 Number of communities for the LFR-128 networks

	GT	*ERKGA*	*OmeGANet*	*ERGA*	*Louvain*	*Infomap*
$\mu = 0.1$	4	4	4	2	5	4
$\mu = 0.2$	4	4	4	2	3	4
$\mu = 0.3$	4	5	4	2	5	7
$\mu = 0.4$	4	6	4	2	7	9
$\mu = 0.5$	4	7	4	2	7	6
$\mu = 0.6$	4	9	4	2	7	9

running a GA maximizing modularity. The provided results show that the combination of kernel similarity and sparsification improves clustering: indeed, the proposed methodology outperforms the other analyzed community detection methods on several networks and is especially suited for dense graphs.

In a first experiment, we investigated 11 different graph kernels to compute effective resistance based similarity choosing this subset from three representative kernel classes: adjacency matrix based, Laplacian matrix based and Markov matrix based. Overall, we conclude that the kernels based on the adjacency matrix of the graph (Gaussian, Exponential, Katz) provide quite stable and good overall performance on the tested real-world networks and on a final experiment analyzing *KOmeGAnet+* over networks with different edge densities, the results indicate that its pre-processing is more effective than our previous works *OmeGAnet* and *OmeGAnet+*, and the Louvain based *LPA* achieving the maximum NMI even cutting links. *K OmeGAnet+* overall demonstrates

- *High performance across sparsity levels*: For sparse networks with $\mu = 0.1$ (density 0.06), *K OmeGAnet+* achieves perfect NMI outperforming or equalling all other methods, including its kernel-enhanced variants.
- *Notable improvements with higher mixing parameters*: as the mixing parameter increases, our method continues to excel. For $\mu = 0.3$, for example, *K OmeGAnet+*

Table 8 NMI results for the LFR-128 networks

	ERKGA	*OmeGANet*	*ERGA*	*Louvain*	*Infomap*
$\mu = 0.1$	1	1	0.3441	0.8793	1
$\mu = 0.2$	1	1	0.3109	0.8716	1
$\mu = 0.3$	0.8894	0.8049	0.1472	0.5555	0.6728
$\mu = 0.4$	0.3791	0.3308	0.0191	0.3228	0.2875
$\mu = 0.5$	0.2123	0.1314	0.134	0.0865	0.1162
$\mu = 0.6$	0.2005	0.0698	0.013	0.055	0.0624

Table 9 Modularity results for the LFR-128 networks

	ERKGA	*OmeGANet*	*ERGA*	*Louvain*	*Infomap*
$\mu = 0.1$	0.7038	0.7038	0.314	0.6268	0.7038
$\mu = 0.2$	0.5446	0.5446	0.2396	0.4661	0.5446
$\mu = 0.3$	0.4112	0.3983	0.165	0.3733	0.4101
$\mu = 0.4$	0.275	0.3308	0.142	0.314	0.2875
$\mu = 0.5$	0.2968	0.2999	0.134	0.3356	0.3386
$\mu = 0.6$	0.2718	0.2854	0.154	0.3086	0.3289

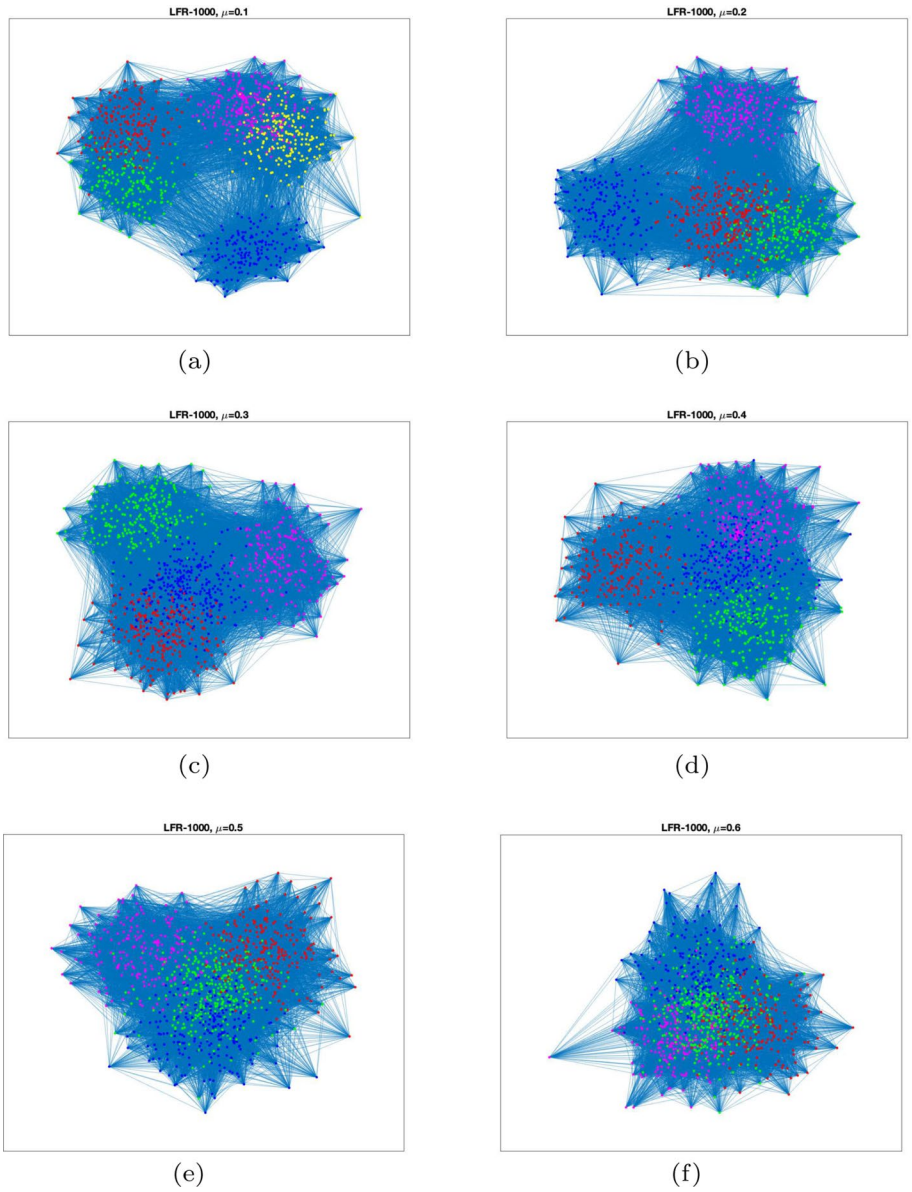

Fig. 10 (a) LFR networks with 1000 nodes generated with mixing parameters values of (**a**) 0.1, (**b**) 0.2, (**c**) 0.3, (**d**) 0.4, (**e**) 0.5 and (**f**) 0.6

Table 11 NMI results for LFR-1000 networks with average edge density 0.06. For *ERKGA*, the exponential kernel is considered

	ERKGA	*OmeGANet*	*ERGA*	*Louvain*	*Infomap*
$\mu = 0.1$	0.7813	1	0.099	1	1
$\mu = 0.2$	0.3657	0.9687	0.019	1	1
$\mu = 0.3$	0.2358	0.6012	0.024	1	1
$\mu = 0.4$	0.092	0.4445	0.002	1	1
$\mu = 0.5$	0.0491	0.266	0.004	1	0
$\mu = 0.6$	0.0251	0.1637	0.002	0.4779	0

achieves NMI 1, showing a vast improvement over *OmeGAnet* (NMI=0.6012).

- *Adaptability to denser networks*: for LFR3 (density 0.49 and $\mu = 0.1$), *K OmeGAnet+* reaches an NMI of 1, outperforming not only *OmeGAnet* and *K OmeGAnet*, but even its non-kernel version, *OmeGAnet+* having NMI 0.9725.

We point out that the proposed method allows to reduce the graph thus resulting in storage gains and computational

Table 12 Modularity results for the LFR-1000 networks

	ERKGA	*OmeGANet*	*ERGA*	*Louvain*	*Infomap*
$\mu = 0.1$	0.5762	0.6864	0.099	0.6864	0.6864
$\mu = 0.2$	0.2345	0.532	0.023	0.5421	0.5421
$\mu = 0.3$	0.1405	0.296	0.026	0.4462	0.4462
$\mu = 0.4$	0.0648	0.1815	0.015	0.3452	0.3452
$\mu = 0.5$	0.0480	0.105	0.014	0.2414	0
$\mu = 0.6$	0.0417	0.073	0.0206	0.1582	0

Table 13 Number of communities for the LFR-1000 networks

	GT	*ERKGA*	*OmeGANet*	*ERGA*	*Louvain*	*Infomap*
$\mu = 0.1$	5	7	5	2	5	5
$\mu = 0.2$	4	7	4	2	4	4
$\mu = 0.3$	4	8	4	3	4	4
$\mu = 0.4$	4	12	4	2	4	4
$\mu = 0.5$	4	11	4	2	4	1
$\mu = 0.6$	4	16	6	2	6	1

Table 14 [NMI, modularity, number of communities] results for types of LFR-1000 networks with different average density: from very sparse (LFR1) to highly dense networks (LFR3). Each network has been generated with $\mu = 0.1$, the kernel used is the Exponential and the number of communities refers to one execution of the method

	Density	*ERKGA*	*OmeGANet*	*ERGA*	*Louv/Infom*
LFR1	0.08	[0.6901, 0.4329, 7]	[1, 0.5576, 3]	[0.038, 0.03, 2]	[1, 0.5576, 3]
LFR2	0.25	[0.6786, 0.4273, 5]	[1, 0.5667, 3]	[0.1333, 0.093, 2]	[1, 0.5667, 3]
LFR3	0.49	[0.6002, 0.2857, 3]	[0.3522, 0.1483, 2]	[0.016, 0.008, 2]	[1, 0.405, 2]

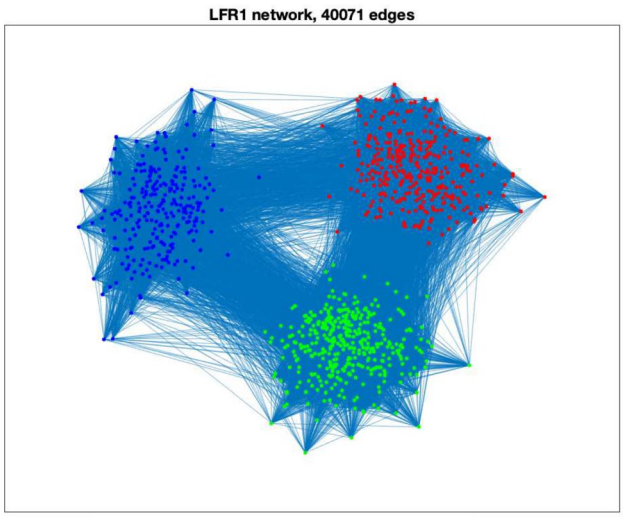

Fig. 11 LFR1 network with 1000 nodes, $\mu = 0.1$ and average density 0.08

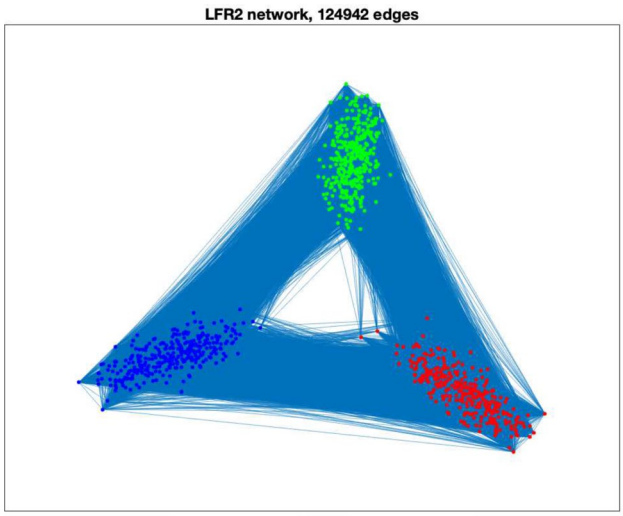

Fig. 12 LFR2 network with 1000 nodes, $\mu = 0.1$ and average density 0.25

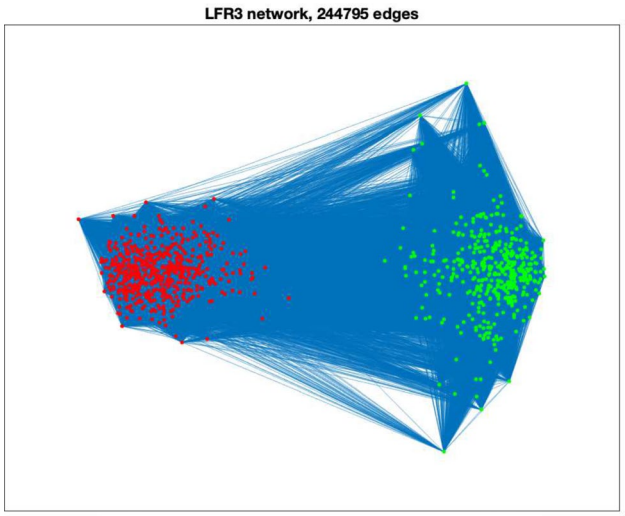

Fig. 13 LFR3 network with 1000 nodes, $\mu = 0.1$ and average density 0.49

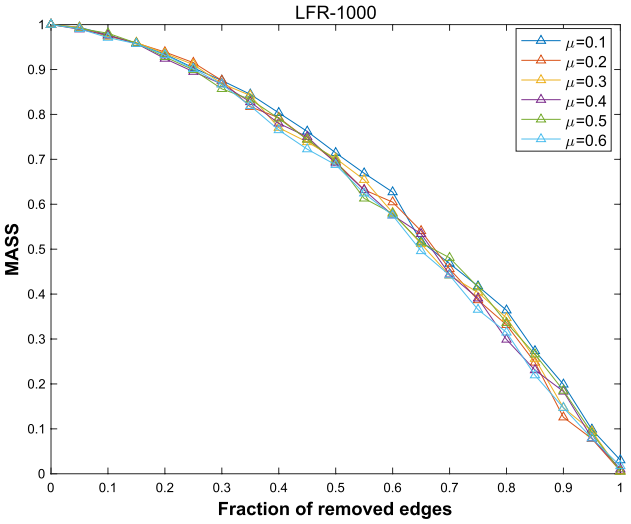

Fig. 14 Variation of MASS for the LFR-1000 networks with density 0.06 and different mixing parameters

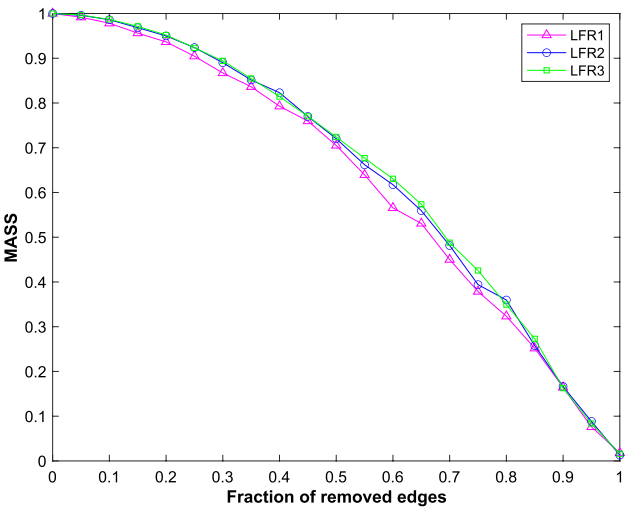

Fig. 15 Variation of MASS for the LFR1, LFR2 and LFR3 networks

time reduction while not loosing the community structure. The main drawback of the scheme, however, is that the procedure to choose the percentage of nodes to remove needs to compute the MASS and the connected components for several possible percentages of edge cuts. This aspect will be investigated in future work by defining a sparsification method that automatically searches the optimal value of edge cuts.

Further work will be also targeted towards kernel transformations Aynulin (2019b) and the possibility of automatically adapting the kernel-parameters as the network size grows. Finally, we also plan to tackle other types of sparsification techniques (e.g. Spielman and Srivastava 1996).

Table 15 [NMI, modularity, number of communities] results for LFR-1000 networks with average edge density 0.06 in terms of NMI, modularity and number of communities. For all the algorithms, the Exponential kernel has been considered

	OmeGAnet	*KOmeGAnet*	*LPA*	*KLPA*	*OmeGAnet+*	*KOmeGAnet+*
$\mu = 0.1$	[1, 0.6864, 5]	[1, 0.6864, 5]	[1, 0.6864, 5]	[1, 0.6864, 5]	[1, 0.6864, 5]	[1, 0.6864, 5]
$\mu = 0.2$	[0.9687, 0.532, 4]	[0.9712, 0.543,4]	[0.683, 0.328, 4]	[0.765, 0.266, 4]	[0.9823, 0.5612, 4]	[1, 0.6864, 5]
$\mu = 0.3$	[0.6012, 0.296, 4]	[0.6423, 0.329,4]	[0.633, 0.332, 4]	[0.723, 0.29, 4]	[0.7453, 0.4749, 5]	[1, 0.6864, 5]
$\mu = 0.4$	[0.4445, 0.1815, 4]	[0.481, 0.2015,4]	[0.574, 0.344, 4]	[0.695, 0.294, 4]	[0.6855, 0.3531, 4]	[1, 0.6864, 5]
$\mu = 0.5$	[0.266, 0.105, 4]	[0.2934, 0.114,4]	[0.497, 0.366, 4]	[0.662, 0.301, 4]	[0.6756, 0.329, 4]	[1, 0.6864, 5]
$\mu = 0.6$	[0.1637, 0.073, 6]	[0.1769, 0.065,6]	[0.374, 0.392, 4]	[0.497, 0.328, 4]	[0.6297, 0.3117, 4]	[0.7023, 0.3912, 4]

Table 16 [NMI, modularity, number of communities] results for types of LFR-1000 networks with different average densities. Each network has been generated with $\mu = 0.1$ and the kernel used is the Exponential

	OmeGAnet	*KOmeGAnet*	*LPA*	*KLPA*	*OmeGAnet+*	*KOmeGAnet+*
LFR1	[1, 0.5576, 3]	[1, 0.5576, 3]	[1, 0.5576, 3]	[1, 0.5576, 3]	[1, 0.5576, 3]	[1, 0.5576, 3]
LFR2	[1, 0.5667, 3]	[1, 0.5667, 3]	[1, 0.5576, 3]	[1, 0.5576, 3]	[1, 0.5667, 3]	[1, 0.5667, 3]
LFR3	[0.3522, 0.1483, 2]	[0.5923, 0.2837, 2]	[0.3423, 0.1256, 2]	[0.5433, 0.2712, 2]	[0.9725, 0.3931, 2]	[1, 0.405, 2]

Acknowledgements We acknowledge the support of the PNRR project FAIR - Future AI Research (PE00000013), Spoke 9 - Green-aware AI, under the NRRP MUR program funded by the NextGenerationEU.

Author contributions All authors contributed to the paper, read and approved the manuscript.

Funding The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Data availability The real-world dataset analyzed during the current study are available in the Figshare repository [https://figshare.com/ndownloader/files/8410949]. The synthetic datasets generated and analysed are not public but will be make available by the authors on request.

Declarations

Competing interests The authors declare that they have no competing interests.

References

Arora R, Upadhyay J (2019) On differentially private graph sparsification and applications. Advances in neural information processing systems 32

Avrachenkov K, Chebotarev P, Rubanov D (2019) Similarities on graphs: Kernels versus proximity measures. European Journal of Combinatorics 80:47–56

Avrachenkov K, Chebotarev P, Rubanov D (2017) Kernels on graphs as proximity measures. In: International Workshop on Algorithms and Models for the Web-graph, pp 27–41. Springer

Aynulin R (2019) Impact of network topology on efficiency of proximity measures for community detection. In: International Conference on Complex Networks and Their Applications, pp 188–197. Springer

Aynulin R (2019) Efficiency of transformations of proximity measures for graph clustering. In: Algorithms and Models for the Web Graph: 16th International Workshop, WAW 2019, Brisbane, QLD, Australia, July 6–7, 2019, Proceedings 16, pp 16–29. Springer

Barrat A, Barthelemy M, Pastor-Satorras R, Vespignan A (2004) The architecture of complex weighted networks. In: Proc. National Academy of Science, pp. 1013747–

Benczúr AA, Karger DR (1996) Approximating st minimum cuts in õ (n 2) time. In: Proceedings of the Twenty-eighth Annual ACM Symposium on Theory of Computing, pp 47–55

Blondel VD, Guillaume J-L, Lambiotte R, Lefebvre E (2008) Fast unfolding of communities in large networks. Journal of statistical mechanics: theory and experiment 2008(10):10008

Chandra AK, Raghavan P, Ruzzo WL, Smolensky R, Tiwari P (1996) The electrical resistance of a graph captures its commute and cover times. Computational Complexity 6(4):312–340

Chebotarev P, Shamis E (2006) On proximity measures for graph vertices. arXiv preprint math/0602073

Chen J, Li J, Bruna J (2021) Deep learning on graphs for node classification: A survey. IEEE Transactions on Neural Networks and Learning Systems 32(1):4–24

Coscia M, Neffke FM (2017) Network backboning with noisy data. In: 2017 IEEE 33rd International Conference on Data Engineering (ICDE), pp 425–436. IEEE

Coscia M, Rossi L (2019) The impact of projection and backboning on network topologies. In: Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, pp 286–293

Dianati N (2016) Unwinding the hairball graph: Pruning algorithms for weighted complex networks. Physical Review E 93(1):012304

Doyle PG, Snell JL (1984) Random Walks and Electric Networks. The Mathematical Association of America, USA

Filippone M, Camastra F, Masulli F, Rovetta S (2008) A survey of kernel and spectral methods for clustering. Pattern recognition 41(1):176–190

Fung WS, Hariharan R, Harvey NJ, Panigrahi D (2011) A general framework for graph sparsification. In: Proceedings of the Forty-third Annual ACM Symposium on Theory of Computing, pp 71–80

Gancio J, Rubido N (2022) Community detection by resistance distance: automation and benchmark testing. In: Complex Networks & Their Applications X: Volume 1, Proceedings of the Tenth International Conference on Complex Networks and Their Applications COMPLEX NETWORKS 2021 10, pp 309–320. Springer

Goldberg DE (1989) Genetic algorithms in search. Optimization, and MachineLearning

Girvan M, Newman ME (2002) Community structure in social and biological networks. Proceedings of the national academy of sciences 99(12):7821–7826

Hamilton WL, Ying R, Leskovec J (2017) Inductive representation learning on large graphs. Advances in Neural Information Processing Systems (NeurIPS) 30:1025–1035 arXiv:1706.02216

Hashemi M, Gong S, Ni J, Fan W, Prakash BA, Jin W (2024) A comprehensive survey on graph reduction: Sparsification, coarsening, and condensation. arXiv preprint arXiv:2402.03358

Jambulapati A, Sachdeva S, Sidford A, Tian K, Zhao Y (2024) Eulerian graph sparsification by effective resistance decomposition. arXiv preprint arXiv:2408.10172

Kim J, Jeong S, Lim S (2022) Link pruning for community detection in social networks. Applied Sciences 12(13):6811

Kipf TN, Welling M (2017) Semi-supervised classification with graph convolutional networks. In: International Conference on Learning Representations (ICLR). arXiv:1609.02907

Klein DJ, Randić M (1993) Resistance distance. Journal of mathematical chemistry 12(1):81–95

Kobayashi T, Takaguchi T, Barrat A (2019) The structured backbone of temporal social ties. Nature communications 10(1):220

Lancichinetti A, Fortunato S, Radicchi F (2008) Benchmark graphs for testing community detection algorithms. Physical review E 78(4):046110

Li J, Zhang T, Tian H, Jin S, Fardad M, Zafarani R (2022) Graph sparsification with graph convolutional networks. International Journal of Data Science and Analytics, 1–14

Li Z, Han Z, Wu S, Wang L, Wang X (2020) Graph neural networks: A review of methods and applications. AI Open 1:57–81

Liu Z, Zhou K, Jiang Z, Li L, Chen R, Choi S-H, Hu X (2023) Dspar: An embarrassingly simple strategy for efficient gnn training and inference via degree-based sparsification. Transactions on Machine Learning Research

Lu B, Liu C, Wang Y (2012) Discovery of community structure in complex networks based on resistance distance and center nodes. J. Comput. Inf. Syst 8(23):9807–9814

Mercier A, Scarpino S, Moore C (2022) Effective resistance against pandemics: Mobility network sparsification for high-fidelity epidemic simulations. PLOS Computational Biology 18(11):1010650

Newman ME (2013) Spectral methods for community detection and graph partitioning. Physical Review E-Statistical, Nonlinear, and Soft Matter Physics 88(4):042822

Park Y, Song M (1998) A genetic algorithm for clustering problems. In: Proceedings of the Third Annual Conference on Genetic Programming, vol. 1998, pp 568–575

Pizzuti C, Socievole A (2021) An effective resistance based genetic algorithm for community detection. In: IJCCI, pp 28–36

Pizzuti C, Socievole A (2021) Effective resistance based weight thresholding for community detection. In: Italian Workshop on Artificial Life and Evolutionary Computation, pp 14–23. Springer

Pizzuti C, Socievole A (2022) Kernels on attributed networks for community detection. In: 2022 13th International Conference on Information, Intelligence, Systems & Applications (IISA), pp 1–8. IEEE

Rosvall M, Bergstrom CT (2008) Maps of random walks on complex networks reveal community structure. Proceedings of the National Academy of Sciences 105(4):1118–1123

Rubido N, Grebogi C, Baptista MS (2013) Structure and function in flow networks. Europhysics Letters 101(6):68001

Saerens M, Fouss F, Yen L, Dupont P (2004) The principal components analysis of a graph, and its relationships to spectral clustering. In: European Conference on Machine Learning, pp 371–383. Springer

Socievole A, Pizzuti C (2023) Community detection in dense networks using effective resistance and link pruning. In: 2023 International Conference on Information and Communication Technologies for Disaster Management (ICT-DM), pp 1–7. IEEE

Socievole A, Pizzuti C (2023) Effective resistance based community detection in complex networks. In: International Conference on Numerical Computations: Theory and Algorithms, pp 197–209. Springer

Sommer F, Fouss F, Saerens M (2016) Comparison of graph node distances on clustering tasks. In: International Conference on Artificial Neural Networks, pp 192–201. Springer

Sommer F, Fouss F, Saerens M (2016) Comparison of graph node distances on clustering tasks. In: Artificial Neural Networks and Machine Learning - ICANN 2016 - 25th International Conference on Artificial Neural Networks, Barcelona, Spain, September 6-9, 2016, Proceedings, Part I, pp 192–201

Sommer F, Fouss F, Saerens M (2017) Modularity-driven kernel k-means for community detection. In: Artificial Neural Networks and Machine Learning - ICANN 2017 - 26th International Conference on Artificial Neural Networks, Alghero, Italy, September 11-14, 2017, Proceedings, Part II, pp 423–433

Spielman DA, Srivastava N (1996) Graph sparsification by effective resistances. Siam Journal on Computing (40), 1913

Spielman DA, Srivastava N (2008) Graph sparsification by effective resistances. In: Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing, pp 563–568

Su Z, Kurths J, Meyerhenke H (2025) Generic network sparsification via hybrid edge sampling. Journal of the Franklin Institute 362(1):107404

Van Mieghem P (2011) Graph Spectra for Complex Networks. Cambridge University Press, New York, NY, USA

Van Mieghem P, Devriendt K, Cetinay H (2017) Pseudoinverse of the laplacian and best spreader node in a network. Physical Review E 96(3):032311

Wang C, Pei J, Bailey J, Zhou Y, Zhang R, Qin L (2019) Dynamic graph neural networks for attributed community detection. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), pp 1229–1237. https://doi.org/10.1145/3292500.3330867

Yan X, Jeub LGS, Flammini A, Radicchi F, Fortunato S (2018) Weight thresholding on complex networks. Physical Review E E98:042304

Yen L, Fouss F, Decaestecker C, Francq P, Saerens M (2007) Graph nodes clustering based on the commute-time kernel. In: Advances in Knowledge Discovery and Data Mining, 11th Pacific-Asia Conference, PAKDD 2007, Nanjing, China, May 22-25, 2007, Proceedings, pp 1037–1045

Yen L, Fouss F, Decaestecker C, Francq P, Saerens M (2009) Graph nodes clustering with the sigmoid commute-time kernel: A comparative study. Data & Knowledge Engineering 68(3):338–361

Yen L, Fouss F, Decaestecker C, Francq P, Saerens M (2009) Graph nodes clustering with the sigmoid commute-time kernel: a comparative study. Data & Knowledge Engineering 68:338–361

Yu S, Alesiani F, Yin W, Jenssen R, Principe JC (2022) Principle of relevant information for graph sparsification. In: Uncertainty in Artificial Intelligence, pp 2331–2341. PMLR

Zhang T, Bu C (2019) Detecting community structure in complex networks via resistance distance. Physica A: Statistical Mechanics and its Applications 526:120782

Zhang Z, Cui P, Zhu W, Zheng Y, Wang F (2019) Attributed graph clustering: A deep attentive embedding approach. In: Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI), pp 4327–4333. arXiv:1906.06532

Zhang G, Sun X, Yue Y, Jiang C, Wang K, Chen T, Pan S (2024) Graph sparsification via mixture of graphs. arXiv preprint arXiv:2405.14260

Zheng Z, Chen X, Lin X (2023) Kernel based dual-channel attributed graph community detection. IEEE Transactions on Network Science and Engineering